University of Khartoum
Faculty of Engineering
Department of Electrical and Electronic Engineering



AnyUp3D: A Temporally Consistent Universal Feature Upsampler for Video Models

A Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of B.Sc. (Honor) in Electrical and Electronic Engineering

Submitted by:

Monzir Hafez **Index: 184102**

Muaaz Abdelmunem **Index: 184100**

Supervisor:

Dr. Magdy Amien

August 2026

DEDICATION

This work is dedicated... To my beloved parents, Hafez and Zubaida, for your endless guidance, and sacrifices that laid the foundation for everything I have achieved.
To my siblings, Mai, Mohammad, Mohanad, Mowaffaq, and Maathir, the ”Passione” group, thank you for always being a pillar of strength and support.
To Abiya, for your steadfast companionship, patience, and the quiet strength you provided through every step of this journey.
And to all my friends who stood by me, especially the members of VOID group,the Hermanos, and the 018 E.E.E. batch, for your motivation, and unwavering support throughout these years.

Monzir

I dedicate this work... To my mother, Nazik, who has been my wise advisor, guiding me through every decision along this journey.
To my father, Abdulmonem, who prepared me to face the hardships of life and taught me its valuable lessons.
To my aunts, who have always prayed, hoped and waited to see this moment come to pass.
To Monzir Hafez, my mate, my friend, and my mentor, for always helping me grow and become a better version of myself.
To VOID, Hermanos, and the EEE 018 family, for being a great source of support, and for making this journey truly memorable.

Muaaz

ACKNOWLEDGMENT

We would like to express our deepest gratitude to Dr. Magdi Amien for staying by our side throughout this journey and for helping us tirelessly whenever we needed him. We are truly grateful for his genuine kindness and his sincere desire to see us succeed, as well as for the valuable advice and constructive criticism he generously offered whenever we needed guidance.

We would also like to express our heartfelt gratitude to our dear seniors, Obada, Ghalib, Wais, Kabbashi, and everyone who stood by us during times of uncertainty, guided us through the early stages of this journey, and helped us take our first steps toward bringing this project to life.

المستخلص

تستخرج نماذج الرؤية التأسيسية مثل "دينو في 2" [1] و"كليب" [2] سمات دلالية قوية، لكن بدقة مكانية منخفضة جدًا، مما يحدّ من استخدامها في مهام التنبؤ الكثيف. وتعالج مُرقّيات السمات مثل "فيت أب" [3] و"لوفت أب" [4] هذه المشكلة باستعادة خريطة سمات عالية الدقة من أخرى منخفضة الدقة دون تشويه معنى السمات، ويقوم النموذج المتطور "إيني أب" [5] بذلك عند أي دقة خرج ومع أي مُرمِّز. غير أن "إيني أب" صُمِّم للصور الثابتة؛ فعند تطبيقه على الفيديو يرقّي كل إطار على حدة ويتجاهل علاقة الإطارات ببعضها عبر الزمن، فيصبح خرجه وميضيًا وتتزحزح حدود الأجسام من إطار إلى آخر. تقترح هذه الأطروحة "إيني أب ثري دي"، وهو امتداد لـ"إيني أب" يضيف الاتساق الزمني مع الحفاظ على قدرته على العمل مع أي عمود فقري. ويُدخل تغييرين على خط أنابيب الترقية: تصبح تلافيف المُرمِّز ثلاثية الأبعاد ليقرأ فترات زمنية قصيرة إلى جانب المكان؛ ويصبح الانتباه المتقاطع ثلاثي الأبعاد، فيستند كل بكسل في الخرج إلى الإطارات المجاورة إضافةً إلى البكسلات المجاورة عند تحديد سمته. دُرِّب "إيني أب ثري دي" على مجموعة جزئية من "يو سي إف-101" (1,170 مقطعًا، 10 أصناف حركية)، بدءًا بالإطارات المفردة ثم بزيادة طول المقطع تدريجيًا. ويستخدم التدريب دالة خسارة رباعية: إعادة البناء مقابل المُرمِّز المعلّم، والتوافق مع الدخل منخفض الدقة، والثبات تحت اضطراب الصورة، وحدّ زمني يعاقب الوميض لكنه يتعطّل حين تختلف الإطارات المتتالية اختلافًا حقيقيًا، حتى لا تُكبَح الحركة الفعلية. وعلى "دافيس-2017"، يرتفع تشابه السمات بين الإطارات المتجاورة من 0.863 إلى 0.944 (+0.081) مقارنةً بالخط الأساس ثنائي الأبعاد القائم على معالجة كل إطار على حدة، مع تحسّن في المقاطع الثلاثين جميعها. وفي اختبار نشر التسميات دون تدريب ودون رأس تجزئة متعلَّم، يرتفع مقياس "جيه آند إف" من 0.266 إلى 0.347 ويتفوّق "إيني أب ثري دي" في 27 مقطعًا من 30 — رغم تطبيقه دون إعادة تدريب على سمات "محوّل سوين للفيديو"، وهو مُرمِّز لم يره أثناء التدريب. يسهم هذا العمل بتقديم أول مُرقٍّ للسمات متسق زمنيًا وغير مرتبط بعمود فقري بعينه ولا بمهمة بعينها، إضافةً إلى خط أنابيب تدريب قابل لإعادة الإنتاج. وقد تحققت هذه النتائج بـ6,000 خطوة تدريب فقط، بعد تهيئة الأوزان انطلاقًا من الأوزان الرسمية لـ"إيني أب تو دي".

ABSTRACT

Vision Foundation Models (VFMs) such as DINOv2 [1] and CLIP [2] extract powerful semantic features, but only at very low spatial resolution, which limits their use in dense prediction tasks. Feature upsamplers such as FeatUp [3] and LoftUp [4] address this by recovering a high-resolution feature map from a low-resolution one without distorting what the features mean, and the state-of-the-art AnyUp [5] does so at any output resolution and for any encoder. AnyUp, however, was built for still images. Applied to video, it upsamples each frame on its own and ignores how frames relate over time, so its output flickers and object boundaries shift from frame to frame. This thesis proposes "AnyUp3D," an extension of AnyUp that adds temporal consistency while keeping its ability to work with any backbone. It makes two changes to the upsampling pipeline: the encoder's convolutions become 3D, so that it reads short spans of time as well as space; and the cross-attention becomes 3D, letting each output pixel draw on nearby frames as well as nearby pixels when deciding its feature. AnyUp3D is trained on a subset of UCF-101 (1,170 clips, 10 action classes), starting with single frames and gradually increasing clip length. Training uses a four-part loss: reconstruction against the teacher encoder, agreement with the low-resolution input, stability under image perturbation, and a temporal term that penalises flicker but switches off when consecutive frames genuinely differ, so real motion is not suppressed. On DAVIS-2017, feature similarity between adjacent frames rises from 0.863 to 0.944 (+0.081) against the frame-by-frame 2D baseline, improving on all 30 clips. In a training-free label-propagation test with no learned segmentation head, the $\mathcal{J}\&\mathcal{F}$ score rises from 0.266 to 0.347 and AnyUp3D wins on 27 of 30 clips, even though it was applied without retraining to features from Video Swin Transformer, an encoder it never saw in training. This work contributes the first temporally consistent feature upsampler tied to neither a specific backbone nor a specific task, along with a reproducible training pipeline. These results come from only 6,000 training steps, after initializing the weights from the original AnyUp2D official weights.

CONTENTS

DEDICATION ii

ACKNOWLEDGMENT iii

CONTENTS ix

LIST OF TABLES x

LIST OF FIGURES xii

LIST OF ABBREVIATIONS xiii

1 INTRODUCTION 1

1.1 Background of the Study 1
1.2 Problem Statement 2
1.3 Significance of the Project 2
1.4 Project Objectives 2
1.5 Scope and Limitations of the Study 3
1.6 Methodology Overview 4
1.7 Organization of the Thesis 4

2 THEORETICAL BACKGROUND AND LITERATURE REVIEW 5

PART I – THEORETICAL BACKGROUND 5
2.1 Transformers and Vision Foundation Models 5
2.1.1 Self-Attention Mechanism 5
2.1.2 Multi-Head Attention 6
2.1.3 Positional Encoding 7
2.1.4 Vision Transformers (ViTs) 7

2.1.5 Vision Foundation Models (VFMs) . . . 8
2.2 Feature Upsampling . . . 9
2.2.1 Interpolation-Based Methods . . . 9
2.2.2 Implicit Neural Representations and LIIF . . . 10
2.2.3 The AnyUp Model . . . 11
2.2.4 Window-Based Attention for Upsampling . . . 11
2.3 Video Understanding and Temporal Modeling . . . 12
2.3.1 Temporal Consistency . . . 12
2.3.2 3D Convolution . . . 12
2.3.3 Temporal Attention . . . 13
2.3.4 Video Object Segmentation (VOS) . . . 13
PART II – LITERATURE REVIEW . . . 14
2.4 Review of Previous Studies . . . 14
2.4.1 Coordinate-Based and Universal Upsampling . . . 15
2.4.2 Feature Refinement and Structural Denoising . . . 16
2.4.3 Video and Temporal Consistency . . . 17
2.5 Comparative Analysis of Previous Work . . . 18
2.6 Research Gap and Project Contribution . . . 19

3 METHODOLOGY 21
3.1 Chapter Overview . . . 21
3.2 Requirements and Specifications . . . 22
3.2.1 Functional Requirements . . . 22
3.2.2 Non-Functional Requirements . . . 23
3.3 Possible Scenarios . . . 23
3.3.1 Training: How to Supervise the Upsampler . . . 23
3.3.2 Training: How to Sample Frames from a Video Clip . . . 24
3.3.3 Architecture: How to Extend the Windowed Attention to the Temporal Domain . . . 25
3.3.4 Architecture: Temporal Window Parameterization — Ratio or Frame Count? . . . 26
3.4 System Framework . . . 26
3.4.1 Stage 1: Input Volume Construction . . . 26
3.4.2 Stage 2: Spatiotemporal Feature Encoding . . . 28
3.4.3 Stage 3: Spatiotemporal Cross-Attention and Upsampling . . . 29

3.5 Dataset Description . . . 31
3.6 Implementation Details . . . 32
3.6.1 Positional Encoding . . . 33
3.6.2 Spatial Reflect 3D Convolution . . . 34
3.6.3 Learned Feature Unification . . . 35
3.6.4 Three-Dimensional Residual Block . . . 36
3.6.5 Spatiotemporal Cross-Attention . . . 37
3.6.6 Training Configuration . . . 39
3.6.7 Loss Function . . . 40
3.7 Evaluation Plan . . . 42
3.7.1 Loss-Based Proxy Metrics . . . 42
3.7.2 Held-Out Reconstruction Evaluation . . . 43
3.7.3 Downstream Evaluation . . . 43
3.8 Chapter Summary . . . 44

4 Experimental Evaluation **46**
4.1 Experimental Setup . . . 46
4.1.1 Hardware and Software . . . 46
4.1.2 Backbone . . . 46
4.1.3 AnyUp3D Configuration . . . 47
4.1.4 Training Details . . . 48
4.1.5 Evaluation Dataset . . . 48
4.1.6 Baselines . . . 48
4.2 Experiment 1: Training Convergence . . . 49
4.2.1 Setup . . . 49
4.2.2 Results . . . 49
4.3 Experiment 2: Temporal Feature Consistency . . . 52
4.3.1 Setup . . . 52
4.3.2 Results . . . 52
4.4 Experiment 3: Downstream Video Object Segmentation . . . 54
4.4.1 Setup . . . 54
4.4.2 Results . . . 56
4.4.3 Analysis of Downstream Results . . . 58
4.5 Experiment 4: Model Size and Inference Cost . . . 59
4.5.1 Setup . . . 59

4.5.2 Results 62
4.5.3 Analysis of Inference Cost 63
4.6 Discussion and Limitations 64
4.6.1 Summary of Findings 64
4.6.2 Limitations 65

5 CONCLUSION AND FUTURE DIRECTIONS 71
5.1 Chapter Overview 71
5.2 Summary of Achievements Against Objectives 71
5.3 Limitations and Critical Reflection 73
5.3.1 Methodological Limitations 73
5.3.2 Validation Limitations 74
5.4 Recommendations for Future Research 74
5.4.1 Short-Term Extensions (1–2 Years) 74
5.4.2 Long-Term Research Avenues (3–5 Years) 75
5.5 Final Concluding Remarks 76

REFERENCES 79

LIST OF TABLES

2.1 Technical Comparison of Feature Upsampling and Video Consistency Architectures 18

4.1 Per-frame cosine similarity between the native 14 × 14 VideoMAE features and the AnyUp3D-upsampled 224 × 224 features (spatially global-average-pooled), comparing the step 250 and step 6,000 checkpoints. Fidelity improves at every frame over the course of training. 51

4.2 Temporal feature consistency (adjacent-frame cosine similarity, higher is better) on DAVIS 2017 validation clips. AnyUp3D improves consistency on every clip, with a mean improvement of +0.081. 67

4.3 Label propagation design decisions for Experiment 3, following established non-parametric video segmentation protocols. 68

4.4 Video object segmentation results on DAVIS 2017 validation (all 30 clips), evaluated via non-parametric label propagation. Scores are weighted per-frame and per-object, following the standard DAVIS evaluation protocol. 68

4.5 Per-clip $\mathcal{J}\&\mathcal{F}$ scores (unweighted), all 30 DAVIS 2017 validation clips. Supplementary to the weighted aggregate in Table 4.4; each clip counted equally regardless of length or object count. 69

4.6 Parameter count and static memory footprint. Buffers are empty in both models, so all static cost comes from learnable weights. 69

4.7 Inference cost on an eight-frame clip at batch size 1, 224 × 224 output resolution, measured over 20 timed iterations after 5 warm-up iterations. The 2D baseline is timed as eight independent per-frame calls; AnyUp3D as a single call on the full volume with q_{chunk} = 50,176. . . 70

5.1 Project objectives and their outcomes. 71

LIST OF FIGURES

2.1 Overview of the Vision Transformer (ViT) architecture. The input image is divided into fixed-size patches, which are flattened and projected into a sequence of patch embeddings. Positional embeddings are added to preserve spatial information, and a learnable classification token is prepended to the sequence. The resulting sequence is processed by a Transformer encoder composed of multi-head self-attention and feed-forward layers [6]. 8

3.1 System framework of the proposed AnyUp3D pipeline, showing the image-guidance path, low-resolution feature path, key aggregation, and the final cross-attention-based upsampling block. 27

4.1 Qualitative comparison of VideoMAE input features and AnyUp3D outputs across training. **Top:** original video frames. **Second row:** PCA visualization of the low-resolution VideoMAE feature maps. **Third row:** AnyUp3D output at step 250. **Bottom:** AnyUp3D output at step 6,000, showing a slight improvement in spatial coherence over the early checkpoint. 50

4.2 Per-frame cosine similarity between the native 14×14 VideoMAE feature and the AnyUp3D-upsampled 224×224 feature, comparing the step 250 and step 6,000 checkpoints (same data as Table 4.1). Fidelity improves at every frame, most notably at frame 7. 51

4.3 Per-clip temporal consistency, 2D AnyUp vs. AnyUp3D, across all 30 DAVIS 2017 validation clips (data from Table 4.2). Every point lies above the $y = x$ diagonal, confirming that AnyUp3D improves temporal consistency on every clip with no exceptions. The four clips with the largest gains are marked. 53

4.4 Per-clip improvement in temporal consistency (Δ = AnyUp3D – 2D AnyUp), sorted in descending order. The largest gains occur on clips with challenging motion (*india*, *motocross-jump*, *bmx-trees*, *dog*, highlighted). 54
4.5 PCA visualization of upsampled features on a representative DAVIS clip. **Top:** 2D AnyUp applied frame-independently, PCA colors shift between frames, indicating temporal inconsistency in the feature space. **Bottom:** AnyUp3D, PCA colors remain stable across frames, with consistent object-to-color assignments throughout the sequence. 55
4.6 Weighted aggregate $\mathcal{J}$, $\mathcal{F}$, and $\mathcal{J}\&\mathcal{F}$ scores, bilinear vs. AnyUp3D, across all 30 DAVIS 2017 validation clips (data from Table 4.4). The gain in $\mathcal{F}$-score is nearly double the gain in $\mathcal{J}$-score. 57
4.7 Per-clip $\mathcal{J}\&\mathcal{F}$, bilinear vs. AnyUp3D, across all 30 DAVIS 2017 validation clips (unweighted, data from Table 4.5). AnyUp3D wins on 27 clips, ties on 1 (*loading*), and loses on 2 (*car-shadow*, *drift-chicane*), labeled. 58
4.8 Qualitative label-propagation results on selected DAVIS 2017 clips. Each group shows the original frame, the propagated mask from bilinear-upsampled features, and the propagated mask from AnyUp3D features. *gold-fish* illustrates AnyUp3D's largest improvement; *drift-chicane*, the one clip with a notable regression, is included for honest comparison. 59
4.9 Per-clip $\mathcal{J}\&\mathcal{F}$ improvement (Δ = AnyUp3D – bilinear), sorted in descending order. The two losses, *car-shadow* and *drift-chicane*, are both small in magnitude relative to the mean gain of +0.074. 60
4.10 Total learnable parameters, 2D AnyUp vs. AnyUp3D (data from Table 4.6). 62
4.11 Inference latency (per eight-frame clip) and peak GPU memory, 2D AnyUp vs. AnyUp3D (data from Table 4.7). AnyUp3D is 2.7× slower and uses 10.6× more peak memory. 63

LIST OF ABBREVIATIONS

CNN	Convolutional Neural Network
DAVIS	Densely Annotated Video Segmentation
mIoU	Mean Intersection over Union
SAM	Segment Anything Model
SOTA	State-of-the-Art
VFM	Vision Foundation Model
ViT	Vision Transformer
VOS	Video Object Segmentation
3D Conv	3D Convolution
HR	High-Resolution
INR	Implicit Neural Representations
LFU3D	Learned Feature Unification (3D)
LR	Low-Resolution
MLP	Multi-Layer Perceptron
MOT	Multi-Object Tracking
NeRF	Neural Radiance Fields
RoPE	Rotary Position Embedding
RoPE3D	3D Rotary Positional Encoding
LiDAR	Light Detection and Ranging
PCA	Principal Component Analysis
VRAM	Video Random Access Memory

Chapter 1

INTRODUCTION

1.1 Background of the Study

Computer vision has witnessed a paradigm shift with the advent of Vision Foundation Models (VFMs) such as DINOv2 [1], CLIP (Contrastive Language–Image Pre-training) [2], and SigLIP (Sigmoid Loss for Language-Image Pre-training) [7]. These models, trained on vast datasets, provide robust semantic representations that are essential for various downstream applications. However, the architecture of these models, primarily Vision Transformers (ViTs), typically produces feature maps at a significantly lower resolution than the input image. For instance, a standard ViT might output a 16×16 feature grid, which lacks the spatial precision required for dense prediction tasks like segmentation or depth estimation. To address this resolution gap, the field of "feature upsampling" has matured. Over the years, feature upsampling evolved from fixed interpolation and unpooling to transposed convolutions, sub-pixel rearrangements, multi-scale pyramids, and content-aware dynamic filtering. Despite improved expressiveness, these approaches remained task-specific and heavily hand-tuned, motivating the emergence of universal, domain-agnostic upsampling models. These learned models, such as the state-of-the-art "AnyUp," treat feature maps as continuous coordinates rather than discrete pixels, allowing for arbitrary resolution scaling without retraining the backbone model. While these advancements have solved the resolution problem for static images, the transition to dynamic video processing introduces new challenges related to temporal stability and consistency.

1.2 Problem Statement

Despite the success of universal feature upsampling in the image domain, a critical gap remains when applying these models to video sequences. Current state-of-the-art universal upsamplers, such as AnyUp and LoftUp, are inherently image-based architectures. Consequently, when applied to video, they operate on a frame-by-frame basis, processing each timestamp independently. Because they were not designed to capture temporal dynamics, they effectively ignore the correlation between consecutive frames. This independence leads to **temporal inconsistency**. Even when individual frames are upsampled with high visual quality, small variations in the input features can cause flickering artifacts, unstable object boundaries, and jittering details over time. There is currently no universal feature upsampling framework that explicitly models temporal consistency, forcing developers to choose between the flexibility of universal upsamplers and the stability of task-specific video models.

1.3 Significance of the Project

This research advances the theoretical framework of universal feature upsampling by extending its reach from the 2D spatial domain into the 3D spatio-temporal domain, effectively bridging the gap between isolated image processing and coherent video understanding. By establishing a foundation for temporally stable feature representations, this approach facilitates high-fidelity, flicker-free semantic selection in video editing and significantly bolsters the precision of Multi-Object Tracking (MOT) and Video Object Segmentation (VOS). Ensuring that high-resolution features remain consistent across timestamps allows perception systems to more accurately detect and follow small objects within complex, dynamic environments, resolving the inherent instabilities found in traditional frame-by-frame processing.

1.4 Project Objectives

The primary goal of this research is to architect a temporally consistent universal feature upsampling model that transcends the limitations of static 2D architectures. By extending the capabilities of the AnyUp framework into the temporal dimension, the project aims to establish an "AnyUp3D" system capable of generating high-resolution, stable feature maps across continuous video sequences. This objective is realized through a

series of technical milestones:

- **Design** a temporally-extended upsampling architecture that incorporates 3D convolutions in the encoder and 3D cross-attention with temporal masking to capture inter-frame dependencies within the AnyUp framework.
- **Construct** the proposed architecture using PyTorch, prioritizing a modular design to ensure the upsampler remains backbone-agnostic and fully compatible with foundational models.
- **Train** the model on the UCF-101 dataset (University of Central Florida, 101 action classes) to cultivate robust temporal priors, allowing the system to learn how features evolve across dynamic motion patterns.
- **Validate** the system's performance using the DAVIS-2017 benchmark (Densely Annotated VIdeo Segmentation), quantitatively assessing spatial precision and temporal consistency through J&F scores (region similarity $\mathcal{J}$ and contour accuracy $\mathcal{F}$) relative to standard frame-wise baselines.
- **Optimize** the inference pipeline to maintain the "universal efficiency" of the original model while accounting for the increased complexity of spatio-temporal data processing.

1.5 Scope and Limitations of the Study

This study focuses specifically on extending the AnyUp model to enhance temporal consistency. The project covers the design and implementation of 3D convolutions in the encoder and 3D cross-attention with temporal masking, and their integration into the upsampling pipeline.

Limitations:

- The study validates the model primarily on the DAVIS-2017 dataset; other video tasks (like depth completion) are outside the primary scope.
- The project does not aim to optimize the model for real-time processing on edge devices; the focus is on accuracy and stability.
- Training is limited by available GPU resources, which may constrain the batch size and sequence length during experimentation.

1.6 Methodology Overview

The methodology follows a structured engineering approach. It begins with a comprehensive theoretical study of existing upsampling and video consistency literature. This is followed by the system design phase, where the "AnyUp3D" architecture is defined, incorporating temporal layers into the existing spatial attention mechanism. The implementation phase involves coding the model in Python/PyTorch and training it on video sequences. Finally, performance evaluation is conducted by benchmarking the proposed model against existing video-based upsampling solutions and by integrating the module into various downstream video architectures to determine if its inclusion yields a measurable improvement in task-specific performance and temporal stability.

1.7 Organization of the Thesis

Chapter One establishes the foundation of the research, providing the background, motivation, and a clear definition of the problem the study intends to solve. **Chapter Two** provides the theoretical background on Vision Transformers, feature upsampling, and video-specific concepts such as Video Object Segmentation, followed by a review of relevant literature. **Chapter Three** details the methodology, system architecture, and implementation steps. **Chapter Four** presents the experimental results, data analysis, and discussion of findings. Finally, **Chapter Five** concludes the work and offers recommendations for future research.

Chapter 2

THEORETICAL BACKGROUND AND LITERATURE REVIEW

PART I: THEORETICAL BACKGROUND

This part establishes the theoretical foundation of the project. It is organised into three groups of concepts that build on one another progressively. The first group covers the Transformer architecture and the large-scale vision models built upon it. The second group covers the principles of feature upsampling, from classical interpolation methods to the state-of-the-art AnyUp model that this project extends. The third group covers the video-specific concepts required to understand temporal consistency and the proposed temporal modules.

2.1 Transformers and Vision Foundation Models

2.1.1 Self-Attention Mechanism

The self-attention mechanism is the fundamental operation underlying the Transformer architecture. Given an input sequence of feature vectors, self-attention computes a new representation for each element by measuring its relationship to every other element in the sequence. This is achieved through three learned linear projections: the Query (Q), the Key (K), and the Value (V) matrices. The Query represents the current element seeking information, the Key represents the elements being compared against, and the Value represents the information to be retrieved.

The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \tag{2.1}$$

where d is the dimension of the key vectors, used as a scaling factor to prevent the dot products from growing too large before the softmax. The softmax produces a probability distribution over all elements, determining how much attention each position should pay to every other. The final output is a weighted sum of the Value vectors, where the elements most relevant to the Query receive the highest weights. This mechanism allows the model to capture long-range dependencies between any two positions in the input sequence, regardless of their distance from each other.

2.1.2 Multi-Head Attention

Multi-head attention extends self-attention by running multiple attention operations in parallel, each with its own independent Q, K, and V projection matrices. Each parallel operation is referred to as an *attention head*. Rather than computing a single attention function over the full feature dimension d, the input is split across h heads, each operating on a lower-dimensional subspace of size d/h. The outputs of all heads are then concatenated and passed through a final linear projection:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \tag{2.2}$$

$$\text{head}_i = \text{Attention}(QW_i^Q,\ KW_i^K,\ VW_i^V) \tag{2.3}$$

where W_i^Q, W_i^K, W_i^V, and W^O are learned projection matrices. The motivation for multiple heads is that different heads can learn to attend to different aspects of the input simultaneously. One head may focus on local spatial structure while another captures global semantic relationships. This diversity of attention patterns makes multi-head attention significantly more expressive than a single attention function, and it is the standard formulation used in all modern Transformer-based architectures, including Vision Transformers, the AnyUp model, and the 3D cross-attention mechanism proposed in this work.

2.1.3 Positional Encoding

The self-attention mechanism described above is permutation-invariant: it has no inherent notion of the order or spatial position of the elements in its input sequence. Since Vision Transformers represent images as sequences of patch embeddings, spatial position information must be explicitly injected into the model before the sequence is processed. This is achieved through *positional encoding*, which adds a position-dependent signal to each patch embedding.

The original Transformer used fixed sinusoidal encodings, where each position is represented by a unique combination of sine and cosine functions at different frequencies. Vision Transformers typically use *learned* positional embeddings instead, where a trainable vector is assigned to each patch position and optimized jointly with the rest of the model. In both cases, positional encoding allows the model to distinguish between patches at different spatial locations even though the self-attention mechanism itself treats all positions symmetrically. This is particularly relevant for feature upsampling, where the precise spatial location of each feature vector determines how it should be mapped to the higher-resolution output grid.

2.1.4 Vision Transformers (ViTs)

Vision Transformers (ViT) apply the Transformer model, originally developed for natural language processing, to computer vision tasks. Instead of processing images through convolutional filters as in Convolutional Neural Networks (CNNs), a ViT represents an image as a sequence of smaller image patches. The input image is divided into fixed-size patches (e.g., 16×16 pixels), and each patch is flattened and projected into a higher-dimensional embedding space. Positional embeddings are then added to these patch representations to preserve spatial information, and the resulting sequence is processed by a Transformer encoder composed of multi-head self-attention layers and feed-forward networks, as described in the preceding subsections.

The output of a Vision Transformer is a set of feature embeddings representing different spatial regions of the image, each containing high-level semantic information. However, because the image is divided into relatively large patches, the resulting feature representation has a significantly lower spatial resolution than the original input image. For a standard ViT with a patch size of $p = 16$, an input image of size $H \times W$ produces a feature grid of only $(H/p) \times (W/p)$ embeddings. While this reduction in resolution improves computational efficiency and encourages the model to focus on seman-

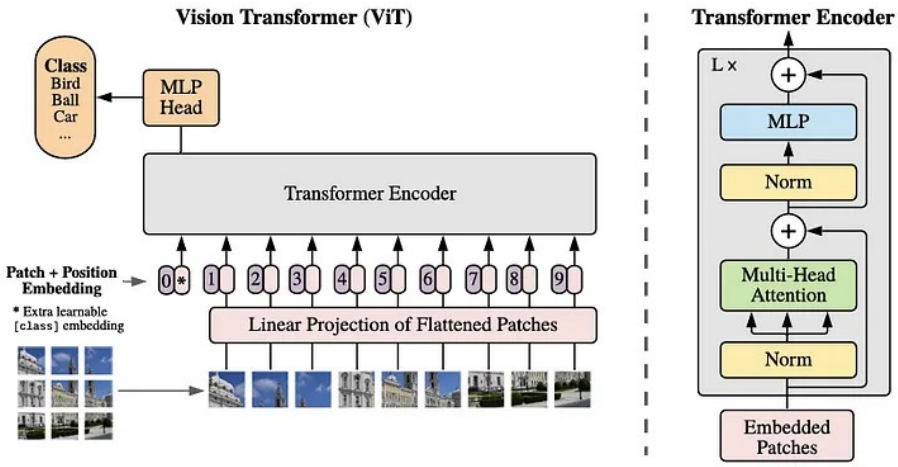


Figure 2.1: Overview of the Vision Transformer (ViT) architecture. The input image is divided into fixed-size patches, which are flattened and projected into a sequence of patch embeddings. Positional embeddings are added to preserve spatial information, and a learnable classification token is prepended to the sequence. The resulting sequence is processed by a Transformer encoder composed of multi-head self-attention and feed-forward layers [6].

tically meaningful patterns, it poses a direct challenge for dense prediction tasks such as segmentation and depth estimation, which require high-resolution spatial outputs. This resolution gap motivates the need for feature upsampling techniques, discussed in the following section.

2.1.5 Vision Foundation Models (VFMs)

Vision Foundation Models are large-scale neural networks trained on massive and diverse datasets to produce general-purpose visual representations. Unlike traditional task-specific models, foundation models learn feature representations that transfer to a wide range of downstream tasks without requiring full retraining. They typically leverage the Vision Transformer architecture and produce rich semantic features that capture high-level information about image content.

Two prominent examples are DINO (self-DIstillation with NO labels) [8] and CLIP [2]. DINO is a self-supervised learning method that trains Vision Transformers without requiring labeled data. It learns meaningful representations by encouraging consistency between different augmented views of the same image through a teacher–student training framework, producing features that are particularly useful for

segmentation and object discovery. CLIP, in contrast, learns visual representations by jointly training on large-scale image–text pairs using contrastive learning. By aligning visual and textual embeddings in a shared representation space, CLIP can perform zero-shot image classification and a wide variety of vision-language tasks. A more recent variant, SigLIP [7], improves on CLIP by replacing the contrastive loss with a sigmoid-based pairwise loss, yielding stronger performance particularly at smaller batch sizes.

Because these models are trained on massive datasets and produce semantically rich features, they have become the dominant choice of feature extractor in modern computer vision pipelines. However, due to their ViT backbone, they all share the same resolution limitation: their output feature maps are produced at a fraction of the input resolution, making feature upsampling an essential post-processing step for any dense prediction task.

2.2 Feature Upsampling

2.2.1 Interpolation-Based Methods

Feature upsampling refers to the process of increasing the spatial resolution of feature maps produced by deep neural networks. The simplest approaches rely on classical interpolation methods, which estimate new values at higher-resolution coordinates from the surrounding low-resolution features.

Nearest Neighbour Interpolation Each upsampled feature is copied from the nearest low-resolution feature:

$$F^{HR}(x, y) = F^{LR}\big(\text{round}(\tfrac{x}{s}),\ \text{round}(\tfrac{y}{s})\big) \tag{2.4}$$

where s is the upsampling scale factor and (x, y) are coordinates in the high-resolution map.

Bilinear Interpolation Each upsampled feature is a weighted sum of the four nearest neighbours:

$$F^{HR}(x, y) = \sum_{i=0}^{1}\sum_{j=0}^{1} w_{ij}\, F^{LR}(x_i, y_j), \qquad w_{ij} = (1 - |x - x_i|)(1 - |y - y_j|) \tag{2.5}$$

Bicubic Interpolation Bicubic interpolation uses sixteen neighbours and a cubic kernel:

$$F^{HR}(x,y) = \sum_{i=-1}^{2} \sum_{j=-1}^{2} R(i,x)\, R(j,y)\, F^{LR}(x_i, y_j) \tag{2.6}$$

where the cubic kernel is:

$$R(t) = \begin{cases} (a+2)|t|^3 - (a+3)|t|^2 + 1, & |t| \leq 1 \\ a|t|^3 - 5a|t|^2 + 8a|t| - 4a, & 1 < |t| < 2 \\ 0, & |t| \geq 2 \end{cases} \tag{2.7}$$

with $a = -0.5$ for the Catmull-Rom spline.

Although these methods are computationally efficient, they do not learn how to recover spatial information lost during downsampling, and they produce blurry or blocky results when applied to semantic feature maps. Learnable approaches such as transposed convolution and sub-pixel convolution (pixel shuffle) address this by training filters to reconstruct spatial detail from compact feature representations. However, all of these approaches remain task-specific and backbone-specific, requiring retraining or retuning for each new architecture. This limitation motivated the development of universal, coordinate-based upsampling methods, described next.

2.2.2 Implicit Neural Representations and LIIF

Traditional feature representations store information as discrete grids of fixed resolution. Implicit Neural Representations (INR) offer an alternative by representing a signal as a continuous function parameterised by a neural network. Given any continuous coordinate as input, the network outputs the corresponding value at that location, allowing the representation to be queried at arbitrary resolutions.

Chen et al. [9] applied this principle to image super-resolution in the Local Implicit Image Function (LIIF). An encoder produces a low-resolution feature grid from the input image. A decoder, implemented as a Multi-Layer Perceptron (MLP), then takes a target coordinate and the features of its surrounding grid cells as input, and outputs the predicted value at that continuous coordinate. Because the MLP is a continuous function, the model can generate outputs at any target resolution during inference, including resolutions not seen during training. This coordinate-based formulation became the conceptual foundation for subsequent universal upsampling models, including AnyUp.

2.2.3 The AnyUp Model

AnyUp is a state-of-the-art universal feature upsampling model proposed by Wimmer et al. [5]. Its primary contribution is the ability to upsample feature maps from any Vision Foundation Model backbone to any target resolution, without task-specific training or manual tuning for each backbone. This backbone-agnostic design is achieved through two key architectural decisions.

First, a Feature-Agnostic Layer normalises the input feature maps to remove statistical biases introduced by different backbone architectures, ensuring that the decoder sees a consistent input distribution regardless of the feature extractor used. Second, a Window Attention module performs the upsampling itself by treating it as a coordinate-based retrieval problem: for each target coordinate on the high-resolution grid, a query is formed from the coordinate embedding and matched against local low-resolution features using scaled dot-product attention.

AnyUp is trained using a self-supervised strategy that requires no manual annotations. Two views of the same image are encoded at different resolutions; the high-resolution view serves as the pseudo-ground-truth, and the model is trained to upsample the low-resolution features to match it using a cosine similarity loss:

$$\mathscr{L} = 1 - \cos(\hat{f}, f) \tag{2.8}$$

where $\hat{f}$ is the predicted high-resolution feature and f is the pseudo-ground-truth. This loss is backbone-agnostic by design, as cosine similarity operates on normalised vectors regardless of their origin.

While AnyUp achieves strong results for static images, it processes video sequences on a frame-by-frame basis, producing temporally inconsistent outputs due to its complete lack of inter-frame communication. This limitation directly motivates the AnyUp3D framework proposed in this thesis.

2.2.4 Window-Based Attention for Upsampling

Rather than applying attention globally across all spatial positions, which is computationally prohibitive for high-resolution feature maps, window-based attention restricts the attention computation to a local neighbourhood around each target location. For a given query position on the high-resolution grid, only the features within a local window of low-resolution patches serve as keys and values. This design dramatically reduces the

computational cost of attention-based upsampling while still providing sufficient local context to recover fine spatial details. Window-based attention is the core mechanism of the AnyUp model and is retained unchanged in the AnyUp3D framework proposed in this thesis.

2.3 Video Understanding and Temporal Modeling

2.3.1 Temporal Consistency

Temporal consistency refers to the property that visual representations or predictions remain stable and coherent across consecutive frames in a video sequence. When a model processes each frame independently, small variations in the input features can cause the output to vary abruptly between adjacent frames, even when the underlying visual content changes only slightly. This leads to temporal artifacts such as flickering, jittering object boundaries, and unstable semantic assignments, all of which degrade the quality of downstream video understanding tasks.

Temporal consistency is enforced by encouraging the model to produce similar representations for temporally adjacent inputs. This can be achieved through several approaches, including temporal loss functions that penalise frame-to-frame variation, motion cues derived from optical flow, and architectures that explicitly model inter-frame dependencies. In this work, temporal consistency is addressed architecturally through the integration of 3D convolutions in the encoder and 3D cross-attention with temporal masking directly into the upsampling pipeline, as described in Chapter 3.

2.3.2 3D Convolution

3D Convolution is a direct extension of the standard 2D convolution operation into the temporal dimension. A 3D convolutional kernel of size $(t \times k \times k)$ slides simultaneously across the temporal dimension and the spatial height and width dimensions of a video volume $(T \times H \times W)$, extracting features that capture both spatial appearance and temporal motion in a single unified operation.

This joint spatial-temporal processing can be more expressive than factorised approaches that apply 2D spatial convolutions and 1D temporal convolutions sequentially, as it can capture correlations between motion and appearance that would otherwise be lost, though at a higher parameter and compute cost. In the proposed AnyUp3D framework, a 3D convolutional layer with a kernel of size 3 × 1 × 1 (temporal × height ×

width) is applied to the stacked feature volume at the early stage of the decoder, before spatial upsampling. The kernel is deliberately restricted to the temporal axis: spatial context is already supplied by the cross-attention mechanism and by the pretrained 2D weights AnyUp3D inherits, so the 3D convolution is needed only to aggregate information across adjacent frames. This keeps the added parameter cost proportional to the temporal extent alone.

2.3.3 Temporal Attention

Temporal Attention is an application of the self-attention mechanism along the temporal axis of a video sequence. Rather than computing attention across all spatial positions within a single frame, temporal attention computes attention across the same spatial position across multiple frames. For a given pixel location (x, y) in the current query frame t, a query vector Q is formed from the feature at that location, while the key matrix K and value matrix V are constructed from the features at the same location across all T frames in the temporal window. The attention score for each frame is computed as the scaled dot product between the query and the corresponding key, followed by softmax normalisation:

$$f_{\text{out}}(x, y, t) = \text{softmax}\left(\frac{Q_{x,y,t} K_{x,y,1:T}^{\top}}{\sqrt{d}}\right) V_{x,y,1:T} \tag{2.9}$$

The resulting output is a weighted combination of features from all frames in the window, where frames whose content most closely resembles the query frame receive the highest attention weights. This allows the model to selectively borrow stable evidence from neighbouring frames to correct inconsistencies in the current frame, directly suppressing flickering artifacts at the feature level.

2.3.4 Video Object Segmentation (VOS)

Video Object Segmentation is a computer vision task that requires identifying and delineating specific objects within each frame of a video sequence while maintaining consistent object identity across the full duration of the clip. The task is typically formulated in a semi-supervised setting, where the ground-truth segmentation mask of the target object is provided for the first frame only, and the model must propagate this mask through all subsequent frames.

VOS is a particularly demanding benchmark for evaluating temporal consistency because accurate segmentation requires stable and coherent feature representations across

frames. Any flickering or instability in the underlying feature maps directly manifests as inconsistent or jittering segmentation boundaries in the output masks. For this reason, VOS is used in this work as the primary downstream evaluation task for the AnyUp3D framework. Performance is measured using the J&F metric, which combines region similarity ($\mathcal{J}$, the Jaccard index) and contour accuracy ($\mathcal{F}$) into a single score, providing a comprehensive measure of both spatial accuracy and boundary sharpness. The evaluation is conducted on the DAVIS-2017 benchmark, which provides densely annotated video sequences covering a wide range of challenging scenarios including fast motion, occlusion, and appearance change.

PART II: LITERATURE REVIEW

2.4 Review of Previous Studies

The evolution of feature upsampling has transitioned from rigid, data-independent heuristics to sophisticated, coordinate-based neural representations. Over the past decade, this progression has been driven by three primary research shifts: first, the move from simple spatial interpolation to content-aware dynamic kernels; second, the transition from task-specific architectures to universal, backbone-agnostic frameworks; and third, the ongoing effort to adapt static image-based models to the temporal demands of video sequences.

To provide a clear roadmap of this technical trajectory, the following literature is categorised into three distinct thematic groups:

1. **Coordinate-Based and Universal Architectures:** Focuses on the recent shift toward treating feature maps as continuous functions rather than discrete grids, enabling arbitrary-scale upsampling across diverse model backbones. This group also covers the key Vision Foundation Models that serve as the feature extractors in these pipelines.

2. **Feature Refinement and Structural Denoising:** Examines methodologies designed to sharpen semantic boundaries and remove artifacts inherent in Transformer-based representations.

3. **Spatio-Temporal Stability:** Explores the specialised techniques developed to maintain visual and semantic coherence across dynamic video frames, includ-

ing 3D convolutions, memory-based attention, and temporal consistency enforcement.

2.4.1 Coordinate-Based and Universal Upsampling

Chen et al. developed the Local Implicit Image Function (LIIF) [9] to overcome the fixed-scale limitations of discrete image representations in super-resolution tasks. By representing images as continuous functions through a Multi-Layer Perceptron that takes latent codes and target coordinates as inputs, the research enabled *infinite resolution scaling*. The study demonstrated that a single model could generalise to arbitrary scales not encountered during training, significantly outperforming traditional interpolation and discrete upsamplers. LIIF established the coordinate-based paradigm that underlies all subsequent universal upsampling models, including those reviewed below.

Fu et al. introduced FeatUp [3], a task-and-model-agnostic framework that restores lost spatial resolution in deep features produced by any Vision Foundation Model backbone. The methodology uses a joint bilateral upsampler that fuses low-resolution semantic features with high-resolution guidance from the original image through a multi-view consistency loss, drawing analogies to Neural Radiance Fields (NeRFs). Two variants are proposed: a fast feedforward upsampler for general use and an implicit per-image model for arbitrary resolution. Crucially, the upsampled features retain their original semantics and can be directly swapped into existing pipelines without retraining. FeatUp demonstrated significant improvements over bilinear interpolation and task-specific upsamplers across class activation mapping, segmentation, and depth prediction tasks. However, like all image-based methods, it processes each frame independently and produces temporally inconsistent outputs when applied to video.

Huang et al. introduced LoftUp [4], a coordinate-based upsampler designed to extract high-resolution features from Vision Foundation Models without manual labels. The methodology utilises a cross-attention transformer to fuse low-resolution semantic tokens with high-resolution geometric cues from the original image. By employing a self-supervised pseudo-ground-truth training signal derived from class-agnostic masks, the model successfully recovered fine-grained spatial details — such as thin object boundaries — that are typically lost in Transformer backbones. LoftUp advances the field by demonstrating that structural information from the original image can guide feature upsampling more precisely than attention alone, though it remains an image-based method with no temporal modeling capability.

Wimmer et al. proposed AnyUp [5], a state-of-the-art universal upsampler designed to function across diverse model architectures without task-specific tuning. The researchers implemented a feature-agnostic window attention that treats upsampling as a localised re-alignment task, mapping features from any backbone onto a higher-resolution grid. While the model proved superior at preserving semantic integrity across diverse backbones, the findings highlighted a critical limitation: its frame-by-frame processing leads to significant temporal instability in video applications, producing flickering artifacts at the feature level. This limitation is the direct motivation for the AnyUp3D framework proposed in this thesis.

Oquab et al. published DINOv2 [1], a family of self-supervised Vision Transformer models trained on a large curated dataset of 142 million images. By combining and scaling several pretraining techniques — including self-distillation and masked image modelling — DINOv2 produces all-purpose visual features that perform competitively with weakly-supervised models across a wide range of vision tasks, including segmentation, depth estimation, and image retrieval, without any fine-tuning. DINOv2 is directly relevant to this thesis as an example of the pretrained feature extractors that the backbone-agnostic AnyUp3D design is compatible with, alongside CLIP and SigLIP. The experiments in Chapter 4 use VideoMAE as the primary feature extractor; DINOv2, CLIP, and SigLIP represent the broader class of Vision Foundation Models that AnyUp3D is designed to support without task-specific retraining.

2.4.2 Feature Refinement and Structural Denoising

Wang et al. introduced CARAFE (Content-Aware ReAssembly of FEatures) [10] to replace standard, content-blind operators such as bilinear interpolation. The authors designed a lightweight kernel prediction module that generates unique upsampling kernels for every spatial location based on local context, allowing for the content-aware reassembly of features. The study proved that adaptive receptive fields significantly boost performance in object detection and instance segmentation by following the actual shapes and textures present in an image. CARAFE established the importance of content-awareness in feature upsampling, but its kernels are generated from a single backbone and cannot be directly applied across different backbone architectures.

Yang et al. explored the inherent artifacts of Vision Transformers in their work on Denoising Vision Transformers [11]. By identifying that positional embeddings often cause “checkerboard noise” in feature maps, they proposed a two-stage denoising

method involving feature decomposition and a learnable denoiser. This research established a vital preprocessing step for cleaning feature maps before upsampling, ensuring that semantic representations remain sharp and free of structural interference. The findings are directly relevant to this project, since the 3D Convolution module in AnyUp3D also receives positional-embedding-corrupted features from the backbone and must process them reliably.

2.4.3 Video and Temporal Consistency

Cheng et al. presented Cutie [12], a video object segmentation network that introduces object-level memory reading to maintain consistency across frames. The model utilises a query-based object transformer to interact with bottom-up pixel features, allowing for high-level semantic tracking while retaining high-resolution feature maps. Their methodology cleanly separated foreground semantics from background distractors, achieving a significant improvement in temporal stability on the MOSE dataset (coMplex video Object SEgmentation) compared to previous pixel-matching baselines. Although Cutie's memory logic is optimised specifically for video object segmentation and cannot be easily applied as a general-purpose upsampler, its object-level temporal memory mechanism serves as an important conceptual reference for the temporally-extended 3D cross-attention proposed in this work.

Liu et al. advocated for spatio-temporal locality in the Video Swin Transformer [13], extending the Swin architecture to the 3D domain. By implementing 3D Patch Partitioning and 3D Shifted Window Attention, the model captures motion characteristics and spatial relationships simultaneously. The key finding was that joint spatio-temporal modeling surpasses factorised models in both efficiency and accuracy, providing a robust backbone for understanding dynamic video content through localised 3D kernels. Video Swin directly validates the design choice of 3D Convolution in AnyUp3D, demonstrating that jointly processing spatial and temporal dimensions at the feature level leads to stronger and more coherent representations than applying spatial and temporal operations sequentially.

Zhou et al. developed Upscale-A-Video [14] to resolve the flickering artifacts common in AI-generated video upscaling. The project integrated 3D Convolutional layers and Temporal Attention modules into a latent diffusion architecture to enforce consistency across the time axis. Their research proved that explicit temporal modeling is essential for maintaining the coherence of moving textures, providing a direct blueprint

for the temporal extensions proposed in this thesis. While Upscale-A-Video is task-specific and computationally expensive owing to its diffusion backbone, it validates the combination of 3D convolutions and temporally-extended cross-attention as the right architectural choices for video feature stabilisation.

Ravi et al. presented Segment Anything Model 2 (SAM 2) [15], a foundation model for promptable visual segmentation in both images and videos. SAM 2 extends the original Segment Anything Model to the video domain by introducing a streaming memory attention module — a lightweight bank of object-level memories drawn from previous frames — that allows the model to track and segment objects consistently across a full video sequence in real time. SAM 2 achieves state-of-the-art results on the DAVIS-2017 benchmark, the same evaluation dataset used in this thesis, establishing it as the strongest contemporary baseline for temporally consistent video segmentation. Crucially, SAM 2's memory mechanism addresses temporal consistency at the *segmentation mask level*, while AnyUp3D addresses the same problem at the more fundamental *feature representation level*, making the two approaches complementary rather than competing.

2.5 Comparative Analysis of Previous Work

The landscape of feature upsampling has evolved through several technical paradigms: from fixed geometric interpolation to content-aware dynamic kernels, and finally toward universal, coordinate-based representations. The primary challenge currently facing the field is what can be termed the *Universality-Stability Gap*: models either provide high backbone flexibility without temporal coherence, or offer temporal stability at the cost of being architecture-locked.

The following table evaluates the key studies reviewed above based on their technical relevance to the development of a temporally consistent, universal upsampler.

Table 2.1: Technical Comparison of Feature Upsampling and Video Consistency Architectures

Model	Domain	Backbone Compatibility	Core Strengths	Technical Weaknesses
LIIF [9]	Image	Backbone-Agnostic	Infinite resolution scaling	Limited to RGB; ignores deep semantics
CARAFE [10]	Image	Task-Specific	Content-aware reassembly	Requires manual backbone tuning
FeatUp [3]	Image	**Backbone-Agnostic**	High-res features, any backbone	Frame-independent; temporal jitter
LoftUp [4]	Image	Backbone-Agnostic	Sharpens VFM boundaries	No temporal modeling
AnyUp [5]	Image	**Backbone-Agnostic**	State-of-the-art semantic retention	**Zero temporal context for video**
Video Swin [13]	Video	Architecture-Locked	Local 3D temporal context	Incompatible with external VFM features
Upscale-A-Video [14]	Video	Task-Specific	High visual fidelity via 3D Conv	High compute; restricted to diffusion
Cutie [12]	Video	Task-Specific	Long-range object memory	Optimised only for segmentation masks
SAM 2 [15]	Video	Task-Specific	Streaming memory; state-of-the-art on DAVIS	Mask-level only; no feature upsampling
AnyUp3D (Ours)	**Video**	**Backbone-Agnostic**	**Universal + temporal consistency**	**Increased spatio-temporal complexity**

Synthesis of Technical Relevance

This comparative study highlights a critical structural void in current upsampling methodologies. Early learned approaches such as *CARAFE* established the superiority of content-aware upsampling over bilinear methods but lacked the backbone compatibility required for the diverse ecosystem of modern Vision Foundation Models. This was addressed by coordinate-based systems such as *LIIF*, *FeatUp*, and *AnyUp*, which treat feature maps as continuous functions and operate across any backbone architecture. While these models excel in modularity and semantic fidelity, their independent frame processing makes them technically insufficient for video, producing spatially accurate but temporally disjointed features that manifest as flickering.

In contrast, dedicated video architectures such as *Video Swin*, *Cutie*, and *SAM 2* provide the necessary infrastructure for temporal stability. *Video Swin* captures local motion through 3D kernels, *Cutie* introduces object-level memory reading, and *SAM 2* brings real-time streaming memory attention for consistent video segmentation. However, these models are fundamentally task-specific or architecture-locked and cannot be applied as general-purpose feature upsamplers for arbitrary backbones such as DINOv2 or CLIP.

AnyUp3D is architected to unify these two paradigms. By incorporating the 3D convolutional spatial-temporal modeling of *Video Swin* and the temporal attention mechanism of *Upscale-A-Video* into the backbone-agnostic framework of *AnyUp* — realised as 3D cross-attention with temporal masking — we resolve the flickering artifacts found in existing universal baselines. This approach provides a first-of-its-kind solution that maintains the flexibility of a Foundation Model upsampler while enforcing the rigorous inter-frame consistency required for high-resolution video understanding.

2.6 Research Gap and Project Contribution

The literature review reveals a clear dichotomy:

- **Universal Upsamplers** (FeatUp, LoftUp, AnyUp) provide flexibility across tasks and backbones but fail to handle the temporal dimension, leading to inconsistent and flickering results in video.
- **Video Consistency Models** (Upscale-A-Video, SAM 2, Cutie) provide stability but are designed for specific tasks and cannot be easily applied as general feature upsamplers across arbitrary Vision Foundation Model backbones.

Project Contribution: This project addresses this gap by proposing *AnyUp3D*. We contribute a novel architecture that integrates the *temporal stability* mechanisms of video models — specifically 3D convolutions in the encoder (validated by *Video Swin*) and 3D cross-attention with temporal masking (validated by *Upscale-A-Video*) — into the *universal framework* of AnyUp. This yields the first task-agnostic, backbone-agnostic feature upsampler capable of producing temporally consistent results for video sequences.

Chapter 3

METHODOLOGY

3.1 Chapter Overview

This chapter presents the engineering methodology followed to design, implement, and evaluate the proposed AnyUp3D framework. AnyUp [5] addresses the low-resolution feature map limitation of Vision Foundation Models (VFMs) such as DINOv2 [1] and CLIP [2] by enabling continuous, backbone-agnostic spatial upsampling. However, when applied to video, AnyUp [5] processes each frame independently, ignoring inter-frame correlations and producing flickering artifacts and unstable object boundaries. AnyUp3D extends AnyUp [5] into the temporal domain by integrating two novel components into its upsampling pipeline: **3D convolutions** inside the encoder to capture spatiotemporal patterns across adjacent frames, and **3D cross-attention with temporal masking** to correct frame-level inconsistencies by attending to neighboring frames within a spatiotemporal window. These additions introduce temporal consistency while preserving the backbone-agnostic universality that defines the original AnyUp [5] design.

The chapter is organized into the following stages:

1. **Requirements and Specifications**, defining the functional and performance criteria the system must meet;

2. **Possible Scenarios**, justifying the chosen architectural approach over alternatives;

3. **System Framework**, presenting the overall three-stage pipeline and data flow, highlighting the novel temporal contributions;

4. **Dataset Description**, documenting the UCF-101 training dataset and the DAVIS-2017 evaluation benchmark;

5. **Implementation Details**, explaining how the model was constructed and integrated; and

6. **Evaluation Plan**, defining the loss-based proxy metrics and held-out validation strategy used to monitor and validate the system's performance.

Each stage is presented with a clear justification of the engineering decisions made to ensure temporal consistency while preserving the universality and backbone-agnostic nature of the original AnyUp [5] design.

3.2 Requirements and Specifications

Based on the problem analysis presented in Chapter One, the following requirements were identified for the proposed AnyUp3D system.

3.2.1 Functional Requirements

- The system must accept a sequence of consecutive video frames alongside their corresponding low-resolution feature maps as input, and produce temporally consistent, high-resolution feature maps as output.

- The system must support arbitrary upsampling scales, preserving the resolution-agnostic behavior of the original AnyUp [5] model.

- The system must reduce temporal inconsistency artifacts (flickering, jittering object boundaries) compared to the frame-by-frame AnyUp [5] baseline, as measured by the mean cosine similarity between adjacent predicted frames on the DAVIS-2017 validation sequences.

- The system must maintain competitive spatial accuracy, measured by the J&F score on DAVIS-2017, relative to the original AnyUp [5] baseline.

3.2.2 Non-Functional Requirements

- **Backbone Agnosticism:** The temporal modules must integrate into the AnyUp [5] pipeline without making any assumptions about the feature extractor used.
- **Scalability:** The model must be trainable on available academic GPU hardware, with the sequence length T and batch size serving as adjustable parameters to accommodate memory constraints.
- **Scope Constraints:** Real-time processing is explicitly out of scope for this project; the primary focus is on accuracy and temporal stability rather than inference speed.

3.3 Possible Scenarios

This section documents three concrete design decisions encountered during the development of AnyUp3D, the alternatives considered for each, and the justification for the choice made.

3.3.1 Training: How to Supervise the Upsampler

Training a feature upsampler requires a source of high-quality supervision. Computing teacher features from arbitrarily high-resolution inputs is computationally infeasible at scale and moves most vision backbones out of distribution. Two practical training strategies from the literature were considered:

- **Option A — JAFAR (Jack up Any Feature At any Resolution) strategy [16]:** Train entirely at low resolutions, where both the supervision signal and the student input are derived from the same input frames without an engineered resolution gap. Couairon et al. argue that low-resolution training is sufficient for learning a generalisable attention-based upsampling mechanism.
- **Option B — AnyUp crop-based strategy [5]:** Sample a high-resolution spatial crop from the input and pass it to the frozen teacher to obtain the feature targets. Simultaneously downsample the same crop to produce a lower-resolution student input. The upsampler learns to recover the teacher's features across this explicit resolution gap.

Decision: Option B was adopted. The AnyUp [5] crop-based strategy was selected because it provides a true high-resolution supervision signal: the teacher's feature targets are extracted from a 224-pixel crop, while the student input is derived from a 112-pixel downsampled version of the same crop, giving the model a non-trivial upsampling objective. This resolution gap forces the upsampler to learn genuine spatial recovery rather than approximate low-resolution feature interpolation. For video, the crop is applied as a consistent spatial tube across all T frames, ensuring that the teacher and student always observe the same scene region and that the temporal supervision signal remains coherent across the clip.

3.3.2 Training: How to Sample Frames from a Video Clip

The temporal sampling interval determines how much real-world motion separates consecutive sampled frames. Too small an interval produces near-identical frames that trivialise the temporal consistency objective; too large an interval produces clips that no longer represent coherent temporal sequences. When constructing a training clip of T frames from a raw video, two strategies were considered:

- **Option A — Dense sampling (every k frames):** Select T consecutive frames with a fixed stride k regardless of the video's frame rate.
- **Option B — Time-based sampling (one frame per t seconds):** Select frames at fixed temporal intervals measured in wall-clock time, so that the sampled clip covers the same real-world duration regardless of the video's frame rate.

Decision: Option B was adopted, with $t = 0.5$ seconds (2 frames per second). Dense sampling on high frame-rate video produces near-identical consecutive frames, making the temporal consistency objective trivially easy and non-generalisable. A fixed frame stride applied to low-frame-rate video skips too much visual content, distorting the motion dynamics of the clip. Anchoring the interval to $t = 0.5$ seconds of wall-clock time maintains a consistent real-world tempo regardless of the source frame rate, giving the temporal modules a meaningful and uniform learning signal throughout training.

3.3.3 Architecture: How to Extend the Windowed Attention to the Temporal Domain

The spatial upsampling core of AnyUp [5] relies on windowed cross-attention, in which each high-resolution query token attends only to key tokens within a local spatial window in the low-resolution feature map. Extending this mechanism to video required a decision on how to incorporate the temporal dimension into the attention receptive field. Three options were considered:

- **Option 1 — 3D attention window:** Extend the existing spatial window with an additional temporal axis, so each query token attends to spatially nearby key tokens within a fixed number of neighboring frames.
- **Option 2 — 2D spatial window with global temporal attention:** Keep the spatial window unchanged but allow each query to attend to all frames simultaneously, removing any temporal locality constraint.
- **Option 3 — Separate spatial and temporal attention modules:** Retain the original 2D windowed attention for spatial upsampling and add a dedicated second attention module that operates purely along the temporal axis.

Decision: Option 1 was adopted. Option 2 introduces quadratic scaling in the temporal dimension: removing the temporal locality constraint multiplies the size of the attention matrix by T, making it prohibitively memory-expensive at moderate resolutions and frame counts. Option 3 avoids this cost but introduces architectural redundancy: two separate attention mechanisms must be trained and maintained, their combined parameters increase the model's footprint, and the interaction between spatial and temporal signals is factorised rather than jointly modelled. The 3D window approach of Option 1 addresses both concerns simultaneously. Temporal locality bounds the attention matrix to a fixed size regardless of clip length, and joint spatiotemporal attention allows the model to reason about spatial detail and temporal context within a single unified operation. It is also the most conservative extension of the existing windowed attention: only the mask computation changes, while the attention layer itself remains unchanged.

3.3.4 Architecture: Temporal Window Parameterization — Ratio or Frame Count?

Having decided to use a 3D attention window, a secondary decision arose regarding how to parameterize the size of the temporal window. Two representations were considered:

- **Option A — Temporal ratio:** Express the temporal window as a fraction of the total clip length T, so the number of attended neighboring frames scales proportionally as T changes.
- **Option B — Absolute frame count:** Express the temporal window as a fixed number of neighboring frames w_t, independent of the total clip length.

Decision: Option B was adopted, with $w_t = 4$. A ratio-based window scales with T: a window of 0.5 covers 4 frames for a clip of $T = 8$ but 8 frames for a clip of $T = 16$. This causes the model's effective temporal receptive field to change as T grows through the curriculum, making the behaviour of the temporal attention inconsistent across training stages. More critically, for long videos with many frames, a fixed ratio forces each query to attend to a large and growing number of temporally distant frames, inflating both the attention matrix and the gradient signal with content that is not temporally relevant to the current frame. An absolute frame count $w_t = 4$ keeps the temporal receptive field constant regardless of clip length, ensuring that the model learns a consistent and transferable notion of neighboring context — from the short clips used in early curriculum stages, to longer clips used later, and to arbitrarily long videos at inference time.

3.4 System Framework

3.4.1 Stage 1: Input Volume Construction

AnyUp3D takes two input streams. The first is a **high-resolution (HR) RGB video** $I \in \mathbb{R}^{B\times 3\times T\times H\times W}$, which supplies the fine-grained structural and temporal guidance needed for high-fidelity upsampling. It is processed by stacked 3D Residual Blocks (Section 3.6.4) followed by **3D Rotary Positional Encoding (RoPE3D)** (Section 3.6.1), which injects relative spatiotemporal position into the token representations by rotating query and key vectors according to their (t, h, w) coordinates.

The second input is a **low-resolution (LR) feature map** $p \in \mathbb{R}^{B\times C\times T\times h\times w}$. Rather than utilizing static image encoders, this volume is extracted from a frozen, pre-trained

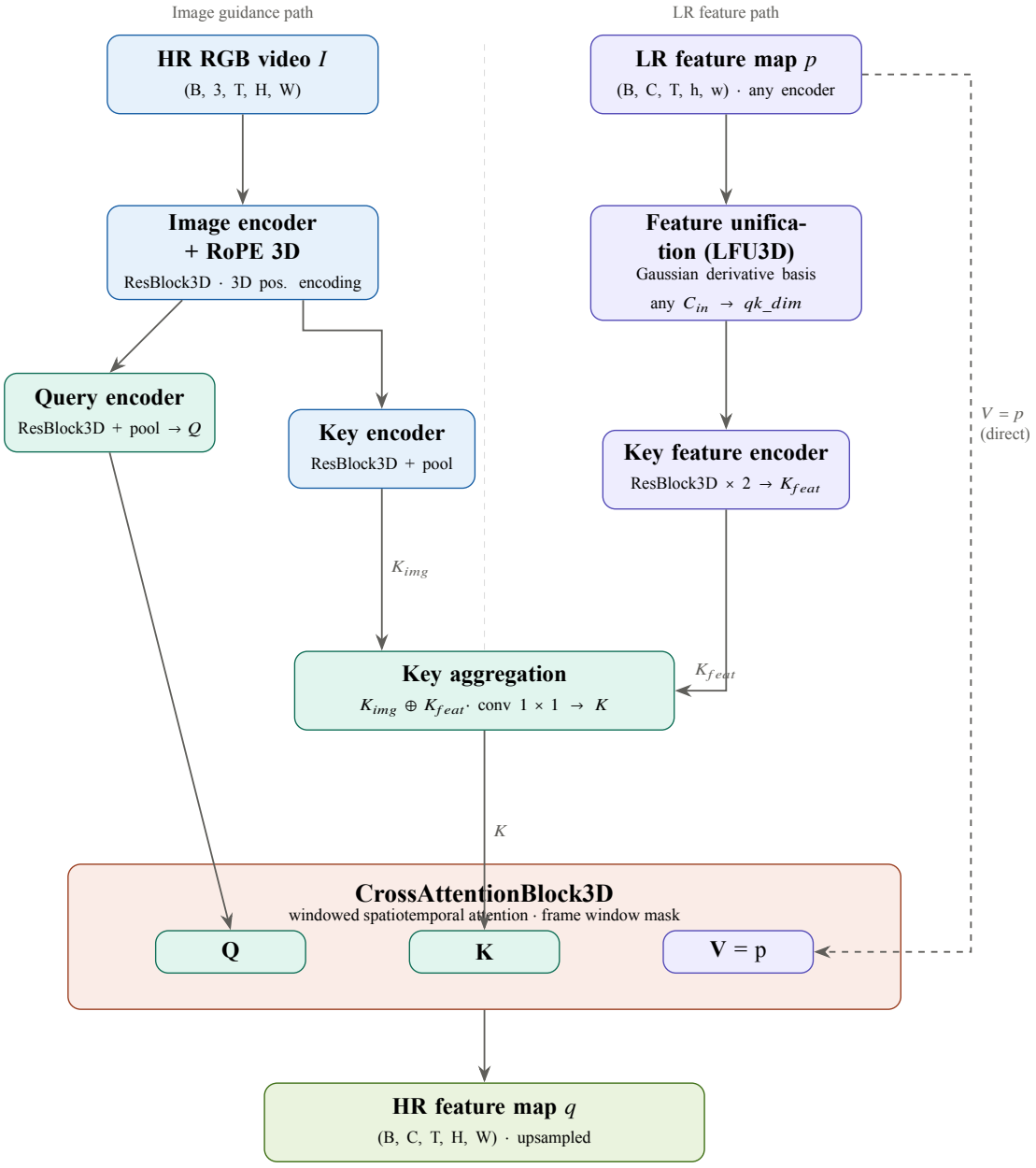


Figure 3.1: System framework of the proposed AnyUp3D pipeline, showing the image-guidance path, low-resolution feature path, key aggregation, and the final cross-attention-based upsampling block.

video processing backbone such as **VideoMAE** or **Video Swin Transformer**. These backbones are uniquely suited to produce feature maps that encapsulate deep semantic information across both spatial and temporal dimensions. Within the AnyUp3D pipeline, these features are passed through a **Feature Unification (LFU3D)** module (Section 3.6.3), the spatiotemporal extension of the Feature-Agnostic Layer used in 2D AnyUp (Section 2.2.3), which employs a learned basis to map arbitrary input channel dimensions to a unified internal representation (qk_dim). While the HR video generates the Queries (Q) and a portion of the Keys (K_{img}), the LR feature map serves both as a source for the key-feature encoding (K_{feat}) and directly as the Value (V) component for the subsequent cross-attention mechanism. All backbone weights are kept frozen throughout training to preserve the backbone-agnostic property of the system.

3.4.2 Stage 2: Spatiotemporal Feature Encoding

Once the input volumes are constructed, the system processes the image guidance and the low-resolution latent features through specialized encoding paths to prepare the Query (Q), Key (K), and Value (V) tensors for cross-attention.

Guidance Path and 3D Rotary Positional Encoding

The high-resolution guidance video I is first processed by a series of 3D Residual Blocks (*ResBlock3D*), which utilize 3D convolutions to extract local spatiotemporal features. To ensure the model is aware of the global spatiotemporal structure, we apply **3D Rotary Positional Encoding (RoPE3D)**. Unlike standard additive positional embeddings, RoPE3D operates by applying a rotation transformation to the feature tokens based on their 3D coordinates (t, h, w).

Specifically, for a feature vector x at a given coordinate, the transformation is defined as:

$$\text{RoPE3D}(x, \text{coords}) = x\cos(\Theta) + \text{rotate_half}(x)\sin(\Theta) \tag{3.1}$$

where Θ is the product of the coordinates and a learned frequency matrix. This approach preserves relative distances between tokens across both space and time, allowing the subsequent attention mechanism to leverage precise geometric and temporal relationships.

Latent Path and Learned Feature Unification (LFU3D)

To maintain a backbone-agnostic architecture, the system must handle latent features p from various video encoders with differing channel dimensions. This is achieved through the **Learned Feature Unification (LFU3D)** module. The LFU3D module employs a depthwise 3D convolution with a learned basis $\mathcal{B} \in \mathbb{R}^{C_{out} \times 1 \times t_k \times k \times k}$.

The module computes an attention-weighted average across the input channels:

$$\text{LFU3D}(p) = \text{mean}_C \left(\text{Softmax}(\text{Conv3D}_{depth}(p, \mathcal{B})) \right) \tag{3.2}$$

This operation ensures that the output is invariant to the input channel dimensionality of the backbone, effectively mapping arbitrary features into a unified qk_dim space. The unified features are then passed through a key feature encoder to form K_{feat}, which is subsequently fused with the image-based keys K_{img} through a concatenation and 1×1 convolution to produce the final Key tensor K.

3.4.3 Stage 3: Spatiotemporal Cross-Attention and Upsampling

The final stage of the AnyUp3D pipeline performs the localized fusion of high-resolution structural cues and low-resolution semantic features. This is achieved through `CrossAttentionBlock3D` (Section 3.6.5), the temporal extension of the Window Attention module used in 2D AnyUp (Section 2.2.3), a windowed spatiotemporal cross-attention mechanism that ensures both spatial detail and temporal consistency in the upsampled output.

Query, Key, and Value Preparation

Before entering the attention mechanism, the encoded representations are projected into a unified embedding space:

- **Queries (Q):** The guidance features are passed through the *query_encoder* and upsampled to the target high-resolution (H, W) using adaptive 3D average pooling. To smooth the upsampled guidance signal and prevent grid artifacts, a $1 \times 3 \times 3$ spatial convolution is applied within the attention block.
- **Keys (K):** The model constructs a composite Key by concatenating image-based keys (K_{img}) with encoded feature keys derived from the normalized latent volume.

These are fused via a *Key Aggregation* layer using a $3 \times 1 \times 1$ convolution, ensuring the Key tensor contains both semantic and structural context.

- **Values (V):** Following a direct bypass design, the original low-resolution feature map p is used as the Value tensor. This preserves the integrity of the backbone's latent representations without additional transformation.

Windowed Spatiotemporal Attention Masking

To manage the quadratic computational complexity of 3D attention, AnyUp3D employs a constrained attention mask. The receptive field of each high-resolution query token is restricted to a 3D local window within the key-value space.

The mask $\mathscr{M}$ is defined by two constraints:

1. **Spatial Constraint:** A spatial window ratio α defines a radius around the projected coordinates of the query. A query at (h_q, w_q) can only attend to keys within the spatial bounds defined by $\pm\alpha$.

2. **Temporal Constraint:** The temporal window w_t restricts attention to neighboring frames. A query at frame t_q only attends to keys at frames t_k where $|t_q - t_k| \leq w_t$.

The resulting boolean mask $\mathscr{M} \in \{0, 1\}^{THW \times Thw}$ is applied to the attention scores, forcing the model to aggregate features from a logically relevant spatiotemporal neighborhood.

Feature Fusion and Final Output

The cross-attention operation uses the prepared Q, K, V and the 3D mask to compute the upsampled feature map $q \in \mathbb{R}^{B \times C \times T \times H \times W}$. By normalizing the queries and keys using *RMSNorm* (Root Mean Square Normalization) and employing multi-head attention, the model effectively "paints" the deep semantic values onto the high-resolution guidance grid.

Notably, the implementation ensures that if $T = 1$ and $w_t = 0$, the 3D temporal logic collapses, making the system mathematically equivalent to the 2D AnyUp [5] baseline. This maintains backward compatibility with image-based backbones while enabling state-of-the-art spatiotemporal upsampling for video-based encoders.

3.5 Dataset Description

This section describes the datasets utilized to develop and evaluate the performance of the proposed AnyUp3D system. To ensure the model achieves both spatiotemporal robustness and high-fidelity reconstruction, a dual-dataset strategy was adopted: the **UCF-101** dataset was used for training, while the **DAVIS-2017** benchmark served as the primary evaluation set.

Training Dataset: UCF-101 (Action Recognition Data)

- **Data Type:** RGB video clips sourced from YouTube, categorized into 101 distinct action classes.
- **Source:** Soomro et al. [17]. It is a widely cited benchmark in the video understanding and computer vision communities.
- **Data Description:** The dataset comprises 13,320 video sequences featuring 101 action categories, such as sports, human-object interaction, and body motions. These clips are characterized by unconstrained environments, varying camera motion, background clutter, and diverse lighting conditions. For this work, training is conducted on a subset restricted to the ten most frequent action classes (approximately 1,170 training and 460 test clips), chosen for manageable size and diversity of motion types.
- **Relevance to This Work:** UCF-101 is selected for the training phase because its immense diversity in motion patterns and real-world "noisy" footage forces the feature upsampler to learn generalized representations. By training on a wide array of temporal dynamics, the AnyUp3D system becomes robust to unpredictable object movements and camera shakes, which is essential for general-purpose feature upsampling.

Evaluation Dataset: DAVIS-2017 (Densely Annotated Video Segmentation)

- **Data Type:** High-resolution (1080p) RGB video sequences with dense, per-frame, per-object segmentation masks for multiple objects per scene.

- **Source:** Pont-Tuset et al. [18], available at the official DAVIS benchmark website (`https://davischallenge.org`).
- **Data Description:** The dataset contains 150 sequences divided into training, validation, and test-dev splits. Each sequence captures a distinct real-world scene featuring complex challenges, including fast motion, motion blur, occlusion, and scale variation. This work utilizes the validation split (30 sequences, ~2,023 frames) for evaluation.
- **Relevance to This Work:** While UCF-101 provides the breadth required for training, DAVIS-2017 provides a rigorous evaluation setting. Its properties support two complementary assessments:
    1. **Spatial Fidelity:** The dense per-frame segmentation masks enable precise measurement of region similarity ($\mathcal{J}$ score) and contour accuracy ($\mathcal{F}$ score).
    2. **Temporal Stability:** The DAVIS-2017 sequences provide a diverse set of real-world video clips across which the temporal stability of upsampled features can be assessed via mean cosine similarity between adjacent predicted frames. This metric operates on the model's own predictions and does not use the DAVIS annotations.
    3. **Fair Comparison:** As the standard benchmark for related architectures such as Cutie and Video Swin Transformer, it enables direct and transparent performance comparison.

3.6 Implementation Details

The proposed system is implemented entirely in Python, using PyTorch as the deep learning framework. The model is designed to operate on five-dimensional tensors of shape (B, C, T, H, W), representing a batch of B video clips each containing T frames of spatial resolution $H \times W$ with C channels. The overall architecture, referred to as AnyUp3D, extends the pretrained 2D AnyUp model [5] into the spatiotemporal domain by introducing a temporal axis across all major computational components, including convolutional layers, positional encodings, and the cross-attention mechanism. A frozen VideoMAE encoder [19], based on a Vision Transformer (ViT-B) pretrained on video data, serves as the teacher model, providing dense spatiotemporal feature targets of shape $(B, 768, T', H', W')$ that supervise the training of AnyUp3D.

The implementation is organized into a set of independent modules, each responsible for a specific component of the architecture. The following subsections describe these modules in order, beginning with the spatiotemporal positional encoding mechanism.

3.6.1 Positional Encoding

Positional information is a critical component in attention-based architectures, as the self- and cross-attention operations are inherently permutation-invariant and carry no built-in notion of spatial or temporal location. The original AnyUp [5] model employs a two-dimensional Rotary Position Embedding (RoPE) [20] to encode the (x, y) spatial coordinates of each token before they enter the cross-attention module. RoPE encodes position by rotating query and key vectors in the embedding space by an angle that is a function of both the token position and the frequency assigned to each embedding dimension, which allows the dot-product attention scores to depend on the *relative* positions of token pairs rather than their absolute coordinates. To support video inputs, this encoding is extended to three axes: temporal (z), horizontal spatial (x), and vertical spatial (y), implemented across two components: `create_coordinates_3d` and `RoPE3D`.

Coordinate Generation. Before positional encoding can be applied, each spatiotemporal token must be assigned a continuous coordinate triple (z, x, y) representing its normalized position along the temporal and spatial axes. The function `create_coordinates_3d` generates these coordinates by constructing three independent linearly spaced grids over the range $[0, 1]$ along the T, H, and W dimensions respectively. A three-dimensional meshgrid is formed from these grids and stacked along the last dimension, producing a coordinate tensor of shape $(1, T \cdot H \cdot W, 3)$ where each row contains the (z, x, y) coordinates of the corresponding spatiotemporal token. Normalizing coordinates to a fixed range ensures that the positional encoding is resolution-independent and behaves consistently when the model is applied to videos of different spatial or temporal sizes.

Rotary Position Embedding. The `RoPE3D` module consumes the coordinate tensor produced by `create_coordinates_3d` and applies rotary positional encoding to the token embeddings. It maintains a learnable frequency matrix $\mathbf{F} \in \mathbb{R}^{3\times d}$, where d is the query/key projection dimension. Each row of $\mathbf{F}$ stores the frequency sequence assigned to one axis, initialized by partitioning the d embedding dimensions into three con-

tiguous non-overlapping blocks and populating each block with a canonical sinusoidal frequency sequence of the form $\theta^{\,\mathrm{linspace}(0,-1,\lfloor \mathrm{block}/2 \rfloor)}$ scaled by 2π, where $\theta = 100$ is the base period. This ensures that each axis operates over a dedicated portion of the embedding space, preventing interference between temporal and spatial position signals at the start of training.

The forward pass computes rotation angles via a single matrix multiplication between the coordinate tensor and the frequency matrix:

$$\mathbf{\Theta} = \mathbf{C} \cdot \mathbf{F} \in \mathbb{R}^{B \times THW \times d} \tag{3.3}$$

The rotated output is then produced by applying the standard RoPE formula:

$$\mathrm{RoPE}(\mathbf{x}) = \mathbf{x} \cdot \cos(\mathbf{\Theta}) + \mathrm{rotate_half}(\mathbf{x}) \cdot \sin(\mathbf{\Theta}) \tag{3.4}$$

where rotate_half($\mathbf{x}$) splits $\mathbf{x}$ along the last dimension into two equal halves $[\mathbf{x}_1, \mathbf{x}_2]$ and returns $[-\mathbf{x}_2, \mathbf{x}_1]$, implementing a 90° rotation in each two-dimensional frequency sub-space. This operation is applied identically and independently to the query and key vectors inside the cross-attention module, ensuring that the resulting attention scores reflect the relative spatiotemporal displacement between each query–key pair.

3.6.2 Spatial Reflect 3D Convolution

A fundamental building block throughout the AnyUp3D architecture is a custom three-dimensional convolutional layer that serves as a direct drop-in replacement for the two-dimensional reflect-padded convolution used in the original AnyUp [5] model. The module, `SpatialReflectConv3d`, wraps a standard `nn.Conv3d` layer and applies padding manually before the convolution, rather than relying on PyTorch's built-in padding argument. This design is necessary because PyTorch's `Conv3d` does not support reflect padding natively, and the padding strategy must be applied asymmetrically across the spatial and temporal dimensions.

The module accepts a kernel size k for the spatial dimensions and an independent temporal kernel size t_k. Spatial padding of $\lfloor k/2 \rfloor$ is applied to the height and width axes using reflect padding, which mirrors the border pixels outward and avoids the introduction of artificial discontinuities at frame boundaries that zero-padding would otherwise cause. Temporal padding of $\lfloor t_k/2 \rfloor$ is applied to the time axis using replicate padding, which repeats the first and last frames rather than reflecting them, a more appropriate boundary condition along the temporal axis, where reflective symmetry has no natural

physical interpretation. When $t_k = 1$, the temporal padding reduces to zero and the convolution operates purely within each frame independently, preserving exact equivalence with the original 2D convolution at inference time when a single frame is processed.

After padding, a single `nn.Conv3d` layer with kernel shape (t_k, k, k) and no built-in padding is applied to the input tensor of shape (B, C, T, H, W), producing an output of the same spatial and temporal resolution. The inner convolution carries no bias term, consistent with the design of the original AnyUp [5] convolutional layers. The module is used throughout the encoder stack wherever a spatial convolution is required, and its temporal kernel size is controlled globally by the training configuration, allowing the degree of temporal mixing to be adjusted without modifying the module itself.

3.6.3 Learned Feature Unification

A key challenge in feature upsampling is aggregating information from an input feature map that may have an arbitrary number of channels into a fixed-dimensional representation suitable for cross-attention. The `LearnedFeatureUnification3D` module addresses this by learning a small bank of basis filters that are applied to every input channel independently, and then combining the responses in a way that produces an output of fixed size regardless of the number of input channels.

The module maintains a single learnable parameter, the basis tensor of shape $(d, 1, t_k, k, k)$, where d is the desired output dimension, k is the spatial kernel size, and t_k is the temporal kernel size. Given an input feature tensor of shape (B, C, T, H, W), the forward pass begins by applying all d basis filters to every one of the C input channels independently via a depthwise 3D convolution. This produces an intermediate tensor of shape $(B, d \cdot C, T, H, W)$, which is then reshaped into (B, d, C, T, H, W) to make the basis and channel dimensions explicit.

A softmax operation is then applied across the basis dimension, producing an attention weight for each of the d filters at every spatiotemporal location. This can be interpreted as the module learning to identify, for each spatiotemporal patch, which of the d basis patterns best describes it. Finally, the result is averaged across the channel dimension C, yielding an output of shape (B, d, T, H, W) that is independent of the number of input channels. This channel-averaging property is what makes the module universal, the same learned basis can process feature maps from encoders of any dimensionality without modification.

To avoid artificially attenuating values near the spatial and temporal boundaries due

to zero-padding, the depthwise convolution is accompanied by a normalization step. A binary mask of ones is padded and convolved with a unit kernel of the same shape as the basis filters, producing a denominator tensor that counts the number of valid kernel cells contributing to each output position. The raw convolution output is divided by this denominator, ensuring that border responses are on the same scale as interior responses.

3.6.4 Three-Dimensional Residual Block

The encoder stack within AnyUp3D is built from stacked residual blocks, which are the primary feature transformation units responsible for progressively refining the intermediate representations before they enter the cross-attention module. The `ResBlock3D` module is a direct three-dimensional extension of the original two-dimensional residual block used in AnyUp [5], with all `Conv2d` layers replaced by their 3D equivalents while preserving the normalization, activation, and skip connection structure unchanged.

Each block consists of a two-layer convolutional sequence. The main branch applies group normalization followed by a SiLU (Sigmoid Linear Unit) activation and a 3D convolution, then repeats this pattern a second time, transforming the input from C_{in} to C_{out} channels. The convolutional layers use a spatial kernel of size k and a temporal kernel of size t_k, and are constructed via a shared factory function that selects between `SpatialReflectConv3d` and a standard `nn.Conv3d` depending on the specified padding mode. When reflect padding is requested, the `SpatialReflect-Conv3d` module described in Section 3.6.2 is used, applying asymmetric reflect and replicate padding manually before the convolution. For other padding modes, a plain `nn.Conv3d` with built-in padding is used instead.

The skip connection adds the input directly to the output of the main branch. When the number of input and output channels differ, or when a convolutional shortcut is explicitly requested, a $1 \times 1 \times 1$ convolution is applied to the input to project it to the correct channel dimension before addition. Because the shortcut kernel has size one, no padding is required and a plain `nn.Conv3d` is always safe to use regardless of the global padding mode. When the channel dimensions already match, the skip connection is an identity mapping. The output of the block is the elementwise sum of the main branch and the shortcut, following the standard residual formulation:

$$\mathbf{y} = \mathscr{F}(\mathbf{x}) + \mathscr{S}(\mathbf{x}) \tag{3.5}$$

where $\mathscr{F}$ denotes the two-layer convolutional branch and $\mathscr{S}$ denotes the shortcut.

Both GroupNorm and SiLU are shape-agnostic operations that act identically on five-dimensional tensors as they do on four-dimensional tensors, requiring no modification for the temporal dimension.

3.6.5 Spatiotemporal Cross-Attention

The cross-attention module is the core of the AnyUp3D architecture. It is responsible for querying the high-resolution image signal with low-resolution feature tokens, allowing the model to recover spatial detail that the feature extractor discards due to its patch-based tokenization. This subsection describes the three components that together implement this mechanism: the attention mask, the attention layer, and the block that wires them into the full pipeline.

Implementation note: Chapter 2 introduces temporal attention as a conceptual simplification, a query at position (x, y) attending to keys at the same position across all T frames (pure column-wise temporal self-attention). The mechanism realised here is different: `CrossAttention3D` implements 3D windowed cross-attention, in which high-resolution query tokens attend to low-resolution key tokens within a local *spatiotemporal* 3D window. This formulation is chosen for efficiency (temporal locality bounds the attention matrix regardless of clip length) and for the richer spatiotemporal context it provides by jointly attending across space and time in a single operation.

Windowed Attention Masking. Applying cross-attention over the full sequence of $T \cdot H \cdot W$ query tokens against $T \cdot H_k \cdot W_k$ key tokens would produce an attention matrix whose size grows quadratically with resolution, making it computationally prohibitive for video inputs. To address this, `compute_attention_mask_3d` constructs a sparse boolean mask that restricts each query token to attend only to a local neighborhood of key tokens, both spatially and temporally.

The spatial restriction reuses the existing 2D windowing logic: for each high-resolution query position, only key tokens whose normalized coordinates fall within a window of radius `spatial_ratio` around the query's position are permitted. The temporal restriction independently limits attention to frames within a temporal distance of `window_t` from the query frame, i.e., a key token at frame t_k is reachable from a query at frame t_q only if $|t_q - t_k| \leq$ `window_t`. When `window_t` is `None`, no temporal restriction is applied and all frames attend to all other frames. The combined spatiotemporal mask is formed by taking the conjunction of the spatial and temporal allowance matrices, yielding a boolean mask of shape $(T \cdot H_q \cdot W_q,\ T \cdot H_k \cdot W_k)$ where

`True` indicates a blocked position. The mask is cached so that it is computed only once per unique combination of resolution and window parameters.

Cross-Attention Layer. The `CrossAttention3D` module wraps PyTorch's `nn.MultiheadAttention` with two additions. First, RMS normalization is applied independently to the query and key sequences before the attention computation, which stabilizes training without introducing learnable scale parameters into the attention scores. Second, the module supports chunked processing of the query sequence: when a chunk size q_{chunk} is specified, the query tensor is split into non-overlapping chunks along the sequence dimension and attention is computed independently for each chunk against the full key and value sequences. The corresponding rows of the attention mask are sliced to match each chunk, and the chunk outputs are concatenated to recover the full output sequence. This reduces the peak memory footprint of the attention matrix from $\mathcal{O}(Q \cdot K)$ to $\mathcal{O}(q_{\text{chunk}} \cdot K)$, which is critical at high spatial or temporal resolutions.

The attention output is not taken directly from `nn.MultiheadAttention`. Instead, the raw attention weight matrix $\mathbf{A} \in \mathbb{R}^{B \times Q \times K}$ is extracted and used to compute a weighted sum over the value sequence explicitly:

$$\mathbf{out} = \mathbf{A} \cdot \mathbf{V}, \quad \mathbf{A}_{ij} = \frac{\exp(\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d})}{\sum_{j'} \exp(\mathbf{q}_i \cdot \mathbf{k}_{j'} / \sqrt{d})} \tag{3.6}$$

This design decouples the key sequence used for computing attention scores from the value sequence used for aggregation, which is necessary because AnyUp [5] uses the low-resolution feature map as both key and value but allows them to be processed differently.

Cross-Attention Block. The `CrossAttentionBlock3D` module is the top-level unit that combines all of the above into a single forward pass. It accepts query, key, and value tensors of shape (B, C, T, H, W). Before attention is computed, a depthwise $1 \times 3 \times 3$ convolution is applied to the query tensor, which mixes spatial context within each frame without mixing across frames and refines the query representation before it is projected into the attention space. The attention mask is then computed from the current spatial and temporal resolution parameters, and all three tensors are flattened from (B, C, T, H, W) into $(B,\ T \cdot H \cdot W,\ C)$ to match the sequence-first format expected by `CrossAttention3D`. After attention, the output sequence is reshaped back into

(B, C, T, H, W).

3.6.6 Training Configuration

Training was conducted entirely within Google Colab using an NVIDIA Tesla T4 GPU (15 GB VRAM). To guard against session disconnects, a practical constraint of the Colab environment, where free-tier sessions expire after approximately 4–5 hours of runtime, all checkpoints and TensorBoard logs were written directly to Google Drive. This setup allowed training to be interrupted and resumed across sessions, with both the model weights and the data-shuffle state restored from the latest checkpoint.

Training Dataset. The model was trained on a subset of UCF-101, restricted to the ten most frequent action classes (approximately 1,170 training and 460 test clips). This subset was chosen for its availability, manageable size, and diversity of motion types. During training, T consecutive frames are sampled at random from each clip and decoded on-the-fly using `PyAV`. Frames are first resized to 256×256 pixels and ImageNet-normalised at the dataset level. A further random spatial tube crop to 224×224 pixels is applied per-sample during the training step, ensuring that the same spatial region is cropped consistently across all T frames. This high-resolution crop (`crop_high`) is passed directly to the frozen VideoMAE teacher to produce the ground-truth feature targets at 14×14 token resolution. It is simultaneously downsampled via bilinear interpolation to 112×112 pixels to produce the low-resolution guidance input (`crop_low`), which AnyUp3D is trained to upsample back to the teacher resolution.

Teacher Model. A frozen VideoMAE encoder (`MCG-NJU/videomae-base`, ViT-B, tubelet size 2, patch size 16) provides the pseudo-ground-truth feature maps that AnyUp3D is trained to reproduce. Given a clip of T frames, the teacher processes the 224×224 high-resolution crop directly, producing spatiotemporal tokens of shape $(B, 768, T/2, 14, 14)$. The same encoder is also applied to the 112×112 low-resolution guidance frames, which it internally upscales to its native 224×224 resolution via bilinear interpolation before computing features; the resulting 14×14 guidance tokens are then further downsampled to 7×7 to serve as the coarse input that AnyUp3D upsamples. Because VideoMAE's positional embeddings are fixed for 16 frames, the temporal positional embedding is bilinearly interpolated to the actual token count $T/2$ before each forward pass and restored immediately afterwards. The encoder is kept frozen throughout; only AnyUp3D is updated.

Temporal Curriculum (T-schedule). Training follows a staged curriculum in which the number of input frames T increases progressively:

Stage	Frames (T)	Batch size	Activates at step
1	2	2	0
2	4	1	5,000
3	8	1	15,000

The minimum of $T = 2$ is imposed by VideoMAE's tubelet embedding, which requires at least one complete temporal tube of two frames. Prior to this curriculum, a brief single-frame ($T = 1$) architectural sanity check was performed by slicing a single temporal token from the VideoMAE feature map; this was used solely to verify that the 2D AnyUp [5] weights loaded correctly into the 3D architecture and that no components were broken, and does not constitute a formal training stage. Batch size is reduced at each curriculum stage to keep GPU memory approximately constant. The curriculum stabilises spatial upsampling quality early in training before the temporal consistency objective is introduced.

3.6.7 Loss Function

Training minimises a four-component objective:

$$\mathscr{L} = \lambda_{\text{rec}}\,\mathscr{L}_{\text{cos-mse}} + \lambda_{\text{inp}}\,\mathscr{L}_{\text{inp}} + \lambda_{\text{aug}}\,\mathscr{L}_{\text{aug}} + \lambda_{\text{temp}}(s)\,\mathscr{L}_{\text{temp}} \tag{3.7}$$

The primary reconstruction term $\mathscr{L}_{\text{cos-mse}}$ is the `Cosine_MSE` criterion from the original AnyUp [5] codebase, applied to the predicted and teacher feature volumes by flattening the temporal and spatial axes before computing the per-pixel cosine similarity and mean squared error. This loss is backbone-agnostic, operating on normalised feature vectors regardless of their absolute scale.

The input consistency term $\mathscr{L}_{\text{inp}}$ enforces that the predicted high-resolution features, when spatially pooled back to the coarse 7×7 resolution, remain consistent with the input guidance features. This discourages the model from producing upsampled features that are spatially plausible but inconsistent with the low-resolution signal it was conditioned on.

The self-consistency term $\mathscr{L}_{\text{aug}}$ encourages robustness to photometric perturbations in the high-resolution guidance signal. An augmentation pipeline applies independent

per-frame brightness and contrast jitter followed by Gaussian noise ($\sigma = 0.02$) to the HR guidance video V, producing a perturbed copy V_{aug}. The coarse feature input p is held fixed and identical across both branches. AnyUp3D is first run with V_{aug} under `torch.no_grad()` to produce a detached pseudo-target $\hat{q}_{\text{aug}}$, and then with the clean V to produce the trainable prediction $\hat{q}_{\text{clean}}$. The loss penalises the cosine-MSE distance between the two outputs:

$$\mathscr{L}_{\text{aug}} = \mathscr{L}_{\text{cos-mse}}(\hat{q}_{\text{clean}}, \hat{q}_{\text{aug}}) \tag{3.8}$$

Gradients flow only through the clean branch. This encourages the model to produce stable feature maps regardless of minor photometric variation in the RGB guidance input.

The temporal consistency term $\mathscr{L}_{\text{temp}}$ penalises frame-to-frame feature differences on stable frame pairs, identified by a binary motion gate derived from the low-resolution guidance frames:

$$\mathscr{L}_{\text{temp}} = \frac{1}{|\mathscr{A}|} \sum_{t \in \mathscr{A}} \mathscr{L}_{\text{cos-mse}}(\hat{f}_t, \hat{f}_{t-1}) \tag{3.9}$$

where $T_t = T / t_{\text{tubelet}}$ is the number of temporal tokens. The active set $\mathscr{A}$ is determined by a hard binary gate:

$$\mathscr{A} = \{t \in [1,\, T_t - 1] : \Delta_t < \tau\}, \quad \Delta_t = \frac{1}{C \cdot H \cdot W} \sum_{c,h,w} \left| I^{\text{low}}_{t,c,h,w} - I^{\text{low}}_{t-1,c,h,w} \right| \tag{3.10}$$

where $\tau = 0.05$ is the RGB-difference threshold. Frame pairs whose mean absolute pixel difference exceeds τ are excluded from the loss entirely; $|\mathscr{A}|$ is the count of active pairs, clamped to at least 1 to prevent division by zero on all-motion clips. The per-pair loss uses the same $\mathscr{L}_{\text{cos-mse}}$ criterion as the reconstruction term, applied to adjacent predicted feature frames. This prevents the loss from penalising legitimate fast motion or scene cuts while still suppressing flickering artifacts in static or slow-moving regions. The term is only active when $T_t > 1$. Its effective weight $\lambda_{\text{temp}}(s)$ is linearly ramped from zero to its final value over the first 5,000 steps of training, giving the model time to stabilise spatial quality before temporal smoothness is enforced. Loss weights are $(\lambda_{\text{rec}}, \lambda_{\text{inp}}, \lambda_{\text{aug}}, \lambda_{\text{temp}}) = (1.0, 0.1, 0.1, 0.2)$.

Optimiser and Learning Rate Schedule. The model is optimised with AdamW (lr = $2{\times}10^{-4}$, weight decay = $1{\times}10^{-4}$). The learning rate decays following a cosine annealing schedule over the full 50,000 training steps, with a minimum of 1% of the peak value.

Gradients are clipped to a maximum ℓ_2 norm of 1.0 to prevent instability at curriculum stage transitions.

Mixed Precision and Initialisation. All forward and backward passes are executed under `torch.bfloat16` mixed precision via `torch.autocast`, with a `GradScaler` handling loss scaling. AnyUp3D weights are initialised from the pretrained 2D AnyUp [5] checkpoint using a weight-adaptation script that centre-initialises newly added temporal kernel dimensions in the 3D convolutions, preserving the single-frame equivalence of the 2D baseline at the start of training.

3.7 Evaluation Plan

The performance of the AnyUp3D system is monitored through two complementary mechanisms: a set of loss-based proxy metrics computed continuously during training, and a periodic held-out reconstruction evaluation on the UCF-101 test split. Together, these provide a principled basis for assessing both training health and model quality throughout the training process.

3.7.1 Loss-Based Proxy Metrics

The four-component loss function defined in Section 3.6.7 serves a dual role: it

provides the training signal but also functions as the primary evaluation instrument during training. Each term measures a distinct and independently interpretable aspect of model quality:

- **Reconstruction quality ($\mathscr{L}_{\text{cos-mse}}$):** The primary indicator of spatial upsampling fidelity. A decreasing reconstruction loss confirms that AnyUp3D is learning to align its upsampled feature maps with the VideoMAE teacher's representations at the target 14×14 resolution. Monitoring this term across curriculum stages also reveals whether spatial quality is preserved as the temporal window T is extended.
- **Input consistency ($\mathscr{L}_{\text{inp}}$):** Validates coherence between the upsampled output and the low-resolution guidance it was conditioned on. A consistently low $\mathscr{L}_{\text{inp}}$ confirms that the model is not hallucinating spatial structure that contradicts the coarse input signal.

- **Self-consistency ($\mathscr{L}_{\text{aug}}$):** Serves as a proxy for robustness. A low and stable $\mathscr{L}_{\text{aug}}$ indicates that predictions are insensitive to small perturbations in the low-resolution guidance features, which is desirable for reliable performance across varied backbone outputs and encoder configurations.

- **Temporal consistency ($\mathscr{L}_{\text{temp}}$):** The primary indicator of temporal stability. Its value should decrease as T increases through the curriculum, confirming that the temporal modules are suppressing frame-to-frame feature fluctuations. The RGB-diff gate ensures that the metric is not penalised in regions of genuine fast motion, making it a meaningful proxy for flickering artifacts rather than a naïve penalty on all inter-frame change.

All four metrics are logged to TensorBoard at every `log_every_n_steps` = 50 steps, alongside the learning rate, curriculum stage T, and batch size. This enables detection of training instabilities, loss divergence at curriculum transitions, and regressions in individual loss components that might otherwise be masked by the total loss.

3.7.2 Held-Out Reconstruction Evaluation

In addition to training-loss monitoring, a held-out reconstruction evaluation is performed on the UCF-101 test split every `val_every_n_steps` = 1,000 training steps. At each evaluation point, the model is switched to eval mode and run over the full test split with the VideoMAE teacher providing ground-truth feature targets. The reconstruction loss $\mathscr{L}_{\text{cos-mse}}$ is computed for each batch, averaged across the split, and logged as `loss/val_rec` to TensorBoard.

This held-out metric serves as the primary overfitting check. If the training reconstruction loss continues to decrease while `val_rec` plateaus or diverges, it signals that the model is memorising specific clips rather than learning generalizable upsampling behavior. Convergence of both training and validation reconstruction loss is the expected signal of a well-behaved model.

3.7.3 Downstream Evaluation

The proxy metrics described above evaluate model quality in terms of feature-space alignment and temporal smoothness. To validate the system's practical value for downstream video understanding tasks, a further evaluation on the DAVIS-2017 validation split is presented in Chapter 4. This evaluation assesses:

- **Spatial fidelity ($\mathcal{J}\&\mathcal{F}$):** Whether the addition of temporal modules preserves the spatial upsampling quality of the original AnyUp [5] baseline, evaluated via region similarity ($\mathcal{J}$) and contour accuracy ($\mathcal{F}$) on video object segmentation.
- **Temporal stability:** Whether the trained model produces coherent, stable feature maps across consecutive frames relative to the frame-by-frame AnyUp [5] baseline, quantified by the mean cosine similarity between adjacent predicted frames on the DAVIS-2017 validation sequences.

3.8 Chapter Summary

This chapter presented the complete methodology for designing and implementing AnyUp3D, a temporally consistent universal feature upsampling framework that extends the state-of-the-art AnyUp model into the spatio-temporal domain.

The chapter began by defining the system's functional and non-functional requirements, establishing backbone agnosticism, modularity, and trainability under limited hardware as the core design constraints. Four design decisions were then considered: the training supervision strategy, where the AnyUp crop-based approach [5] was chosen over JAFAR [16] for its explicit resolution gap; the frame sampling strategy, where time-based sampling ($t = 0.5$s) was preferred over fixed-stride sampling for consistent motion tempo; the extension of windowed attention to video, where a unified 3D attention window was adopted over global or separate temporal attention for its bounded cost and joint spatiotemporal modelling; and the temporal window parameterization, where an absolute frame count ($w_t = 4$) was chosen over a ratio-based scheme for consistency across curriculum stages.

The system framework was subsequently presented as a three-stage pipeline. **Stage 1 (Input Volume Construction)** assembles the high-resolution RGB video and low-resolution feature volumes from the frozen backbone into the tensors required by the decoder. **Stage 2 (Spatiotemporal Feature Encoding)** processes both volumes through stacked ResBlock3D units, which incorporate 3D convolutions to extract local spatiotemporal patterns; this stage also applies 3D Rotary Positional Encoding to the guidance path and the Learned Feature Unification module to normalize arbitrary backbone features into a fixed-dimensional space. **Stage 3 (Spatiotemporal Cross-Attention and Upsampling)** performs the core upsampling via `CrossAttentionBlock3D`, which implements 3D windowed cross-attention, high-resolution

query tokens attending to low-resolution key tokens within a local spatiotemporal window. Temporal consistency is realized inside this cross-attention block through the 3D mask rather than via a separate standalone component.

The UCF-101 dataset was documented as the training source, and DAVIS-2017 was identified as the benchmark for downstream evaluation presented in Chapter 4. Implementation details were subsequently specified, covering the PyTorch-based modular architecture, the $3 \times 1 \times 1$ convolution kernel, the four-component loss function with RGB-diff-gated temporal consistency term, and the memory-efficient training adaptations adopted for the Google Colab environment.

Finally, the evaluation plan was defined across two levels: a training-time evaluation using the four loss components as proxy metrics for spatial fidelity, input coherence, robustness, and temporal stability—monitored continuously via TensorBoard—and a periodic held-out reconstruction evaluation on the UCF-101 test split to guard against overfitting. The downstream evaluation on the DAVIS-2017 validation split, measuring $\mathcal{J}\&\mathcal{F}$ and temporal feature stability relative to the frame-by-frame AnyUp [5] baseline, is presented in Chapter 4.

The next chapter will present the experimental results obtained from applying this methodology, including the quantitative performance metrics observed during training and a discussion of the model's convergence behavior across the temporal curriculum stages.

Chapter 4

EXPERIMENTAL EVALUATION

This chapter presents the experimental evaluation of AnyUp3D. Section 4.1 describes the experimental setup, including hardware, backbone, model configuration, and training details. Section 4.2 provides a qualitative and quantitative analysis of training convergence by comparing early and late checkpoints. Section 4.3 evaluates temporal feature consistency against the frame-independent 2D AnyUp baseline across the full DAVIS 2017 validation set. Section 4.4 measures the downstream impact of AnyUp3D features on video object segmentation using the standard $\mathcal{J}\&\mathcal{F}$ metric. Section 4.5 quantifies the computational price of the temporal extension, benchmarking parameter count, inference latency, and peak GPU memory against the frame-independent 2D baseline. Finally, Section 4.6 discusses the results in aggregate, examines the two clips where AnyUp3D underperforms bilinear, and acknowledges the evaluation's limitations.

4.1 Experimental Setup

4.1.1 Hardware and Software

All experiments were conducted on Google Colab. The implementation uses PyTorch 2.x with CUDA, and models were run in mixed-precision (`float16`) where supported.

4.1.2 Backbone

We use VideoMAE [19] (ViT-B/16, pretrained on Kinetics-400 [21]) as the primary spatiotemporal feature extractor and as the backbone AnyUp3D is fine-tuned against dur-

ing training. VideoMAE processes 16 input frames using a tubelet size of 2, producing $T' = 8$ temporal tokens. Each frame is patchified at 16×16, yielding a spatial resolution of 14×14 per temporal slice. The resulting feature tensor has shape $(1, 768, 8, 14, 14)$, that is, 768-dimensional features at $8 \times 14 \times 14$ spatiotemporal positions.

We chose VideoMAE because it produces genuinely spatiotemporal tokens: its tubelet embedding jointly encodes pairs of adjacent frames, so the features at each position already carry short-range temporal context. This makes it a natural input for AnyUp3D, which extends this temporal structure to the upsampled resolution.

VideoMAE is used for Experiments 1 and 2 (Sections 4.2, 4.3). Experiment 3 (Section 4.4) additionally evaluates AnyUp3D on Video Swin-T features, a backbone distinct from VideoMAE, as a deliberate zero-shot generalization test described there.

4.1.3 AnyUp3D Configuration

3D Convolutions. AnyUp3D replaces the 2D depth-wise convolutions in the value projection with 3D convolutions using a kernel of size $3 \times 1 \times 1$ (temporal × height × width) — that is, a temporal extent of $t_k = 3$ over a 1×1 spatial footprint. This allows each feature position to directly aggregate information from its immediate temporal neighbors during the projection step, without requiring the attention mechanism to span across time, and without enlarging the spatial receptive field inherited from the 2D model.

3D Spatiotemporal Attention. The cross-attention window covers a local spatial neighbourhood determined by a window ratio of 0.1 relative to the spatial resolution, matching the 2D AnyUp default. Along the temporal axis the window spans $w_t = 4$ adjacent slices, chosen as an absolute frame count (rather than a resolution-relative ratio) to keep the receptive field consistent across T-curriculum stages, as detailed in Chapter 3.

Additional hyperparameters, query/key dimension $d_{qk} = 128$, four attention heads, match the 2D AnyUp defaults. Together the 3D convolutions capture local short-range temporal continuity, while the 3D attention enables the model to align spatiotemporal features with the high-resolution RGB guide.

4.1.4 Training Details

AnyUp3D was trained for 6,000 steps using the AdamW optimizer with a learning rate of 2×10^{-4}. Model weights were initialised from the pretrained 2D AnyUp checkpoint, providing a strong spatial upsampling prior before any temporal training begins. Training followed the T-curriculum strategy described in Chapter 3: beginning with $T = 2$ frames to warm up the spatial upsampling pathway, then increasing to $T = 4$ at step 5,000. The third stage ($T = 8$, scheduled to activate at step 15,000) was not reached within the current training run; completing this stage is left as future work that would expose the model to the full temporal resolution used at evaluation time. The objective function combines four loss terms: $\mathscr{L}_{\text{cos-mse}}$, $\mathscr{L}_{\text{input-consistency}}$, $\mathscr{L}_{\text{self-consistency}}$, and $\mathscr{L}_{\text{temporal-consistency}}$ with RGB-difference gating, as detailed in Section 3.6.7 of Chapter 3.

4.1.5 Evaluation Dataset

All evaluations use the DAVIS 2017 validation set [18], a standard video object segmentation benchmark containing 30 video clips with pixel-level ground-truth annotations. DAVIS 2017 is well suited for this evaluation: its clips are short enough to process on a T4 GPU, cover diverse motion types (from slow camera pans to fast athletic movements), and provide ground-truth masks that enable both quantitative and qualitative assessment.

4.1.6 Baselines

We compare AnyUp3D against two baselines:

Bilinear upsampling. The VideoMAE feature tensor at 14×14 spatial resolution is upsampled to 224×224 using trilinear interpolation, independently per channel. This baseline requires no learned parameters and represents the simplest possible spatial upsampling strategy.

2D AnyUp (frame-independent). The pretrained 2D AnyUp model [5] is applied independently to each frame. Each temporal slice of the VideoMAE feature tensor is treated as a separate image-level feature map and upsampled using the original AnyUp architecture with no temporal context. This baseline isolates the contribution of the temporal extension: any improvement of AnyUp3D over this baseline is attributable to

the 3D temporal modeling rather than the spatial cross-attention mechanism, which both models share.

4.2 Experiment 1: Training Convergence

Before evaluating AnyUp3D on benchmark metrics, we verify that the model learns meaningful spatial structure during training. We do so by comparing the upsampled features produced by an early checkpoint (step 250) against the final checkpoint (step 6,000), applied to the same input video.

4.2.1 Setup

A short video clip is processed through VideoMAE to produce features of shape $(1, 768, 8, 14, 14)$. Both checkpoints receive the same low-resolution features and the same RGB guide video, and produce upsampled output at 224×224 spatial resolution. We visualize the output using PCA (Principal Component Analysis): the top three principal components of the 768-dimensional feature vectors are mapped to RGB channels, with PCA fitted jointly across all frames to ensure color consistency over time.

In addition to this qualitative visualisation, we compute a quantitative feature-fidelity metric. For each frame, we spatially average-pool both the native 14×14 low-resolution VideoMAE feature map and the 224×224 AnyUp3D output down to a single 768-dimensional vector, and measure the cosine similarity between the two. This captures whether the 16× spatial upsampling preserves the semantic content of the original feature, independent of pixel-level spatial detail.

4.2.2 Results

Figure 4.1 shows the comparison. Both checkpoints produce spatially reasonable PCA visualisations, which is expected: the model is initialised from pretrained 2D AnyUp weights, so it starts from a non-trivial spatial upsampling quality. The step 6,000 output is slightly more coherent than step 250, with colours that correspond somewhat better to the visible scene structure. The key takeaway is that training does not hurt the model, the 3D extension leaves the inherited spatial quality intact and nudges it slightly forward, confirming that the training procedure is on the right track.

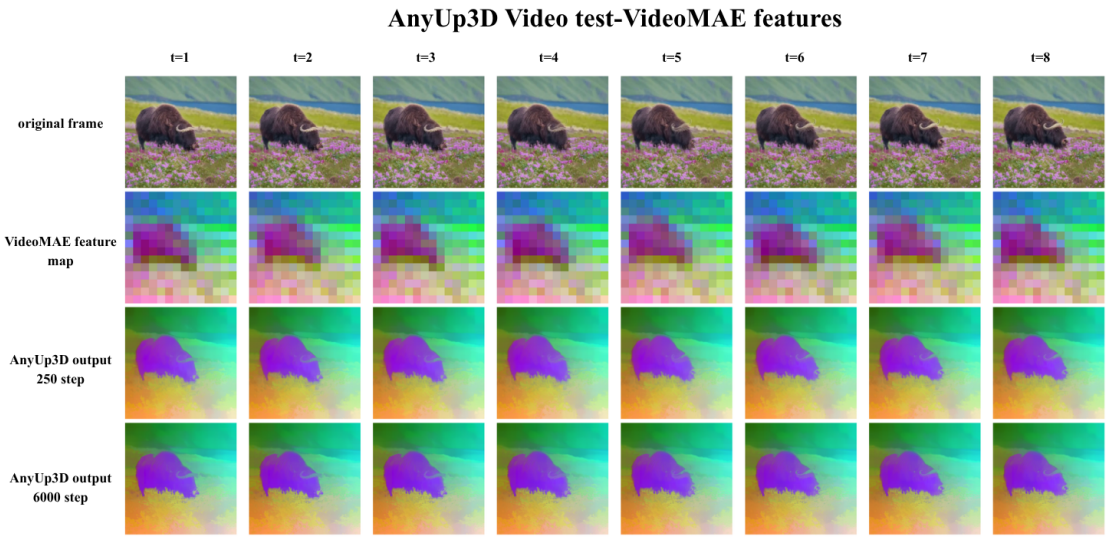


Figure 4.1: Qualitative comparison of VideoMAE input features and AnyUp3D outputs across training. **Top:** original video frames. **Second row:** PCA visualization of the low-resolution VideoMAE feature maps. **Third row:** AnyUp3D output at step 250. **Bottom:** AnyUp3D output at step 6,000, showing a slight improvement in spatial coherence over the early checkpoint.

Quantitative Feature Fidelity. Table 4.1 reports the per-frame cosine similarity between the native 14×14 VideoMAE feature and the 224×224 AnyUp3D output, computed at both the step 250 and step 6,000 checkpoints on the same clip shown in Figure 4.1.

Fidelity is already high at step 250, a direct consequence of initialising from the pretrained 2D AnyUp weights, which already learned to upsample without distorting feature identity, and improves further, on every single frame, by step 6,000, with the mean cosine similarity rising from 0.9899 to 0.9913. The improvement is modest in absolute terms, but this is expected: because fidelity starts near ceiling, there is limited room left for training to improve it further. The direction and consistency of the change is nonetheless informative, mirroring the all-clips-improve pattern later observed in the temporal consistency results of Experiment 2 (Section 4.3), and confirms that additional training refines rather than degrades the spatial feature quality inherited from the 2D initialisation. This result is consistent with, and quantitatively strengthens, the qualitative PCA comparison above: the model preserves feature identity while increasing spatial resolution, which is a necessary (though not sufficient) condition for AnyUp3D to serve as a drop-in replacement for bilinear upsampling.

Note that, like the PCA comparison, this fidelity check is computed on a single

Table 4.1: Per-frame cosine similarity between the native 14 × 14 VideoMAE features and the AnyUp3D-upsampled 224×224 features (spatially global-average-pooled), comparing the step 250 and step 6,000 checkpoints. Fidelity improves at every frame over the course of training.

Frame	**Step 250**	**Step 6,000**	**Δ**
0	0.9921	0.9938	+0.0017
1	0.9889	0.9915	+0.0026
2	0.9912	0.9918	+0.0006
3	0.9911	0.9916	+0.0005
4	0.9889	0.9896	+0.0007
5	0.9874	0.9885	+0.0011
6	0.9872	0.9886	+0.0014
7	0.9921	0.9952	**+0.0031**
Mean	0.9899	0.9913	**+0.0014**

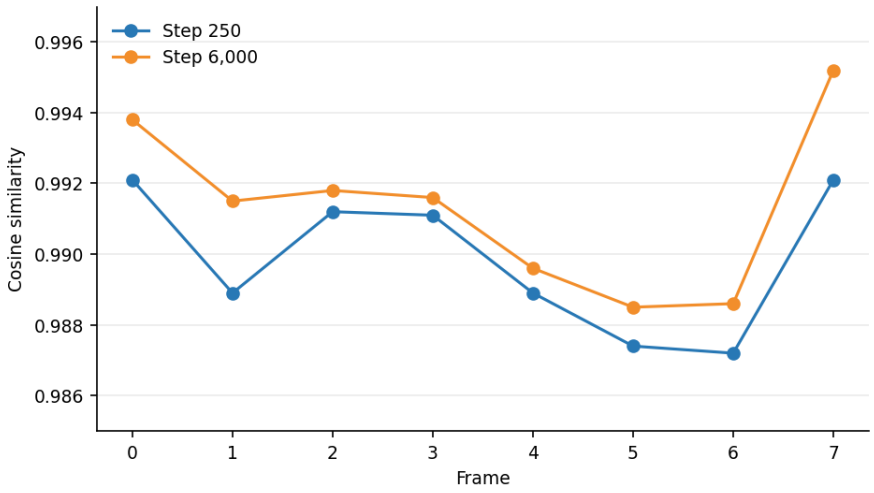


Figure 4.2: Per-frame cosine similarity between the native 14 × 14 VideoMAE feature and the AnyUp3D-upsampled 224 × 224 feature, comparing the step 250 and step 6,000 checkpoints (same data as Table 4.1). Fidelity improves at every frame, most notably at frame 7.

representative clip rather than averaged across the DAVIS 2017 validation set as in Experiments 2 and 3, and should be read as a targeted diagnostic rather than a benchmark result.

4.3 Experiment 2: Temporal Feature Consistency

The central claim of AnyUp3D is that jointly processing the temporal dimension produces more temporally coherent features than applying a 2D upsampler independently to each frame. This experiment tests that claim directly.

4.3.1 Setup

For each of the 30 video clips in the DAVIS 2017 validation set, we extract VideoMAE features and upsample them using two methods: (1) 2D AnyUp applied independently to each frame, and (2) AnyUp3D applied to the full spatiotemporal volume. We then measure temporal consistency as the mean cosine similarity between the feature maps of adjacent frames. Concretely, for an output tensor of shape $(1, C, T', H, W)$, we compute:

$$\text{TemporalSim} = \frac{1}{T' - 1} \sum_{t=0}^{T'-2} \cos(\mathbf{q}_t, \mathbf{q}_{t+1}), \tag{4.1}$$

where $\mathbf{q}_t \in \mathbb{R}^{C \cdot H \cdot W}$ is the flattened feature map at time step t. Higher values indicate greater temporal stability, that is, features that change smoothly between adjacent frames rather than flickering.

4.3.2 Results

Table 4.2 reports per-clip cosine similarity for all 30 DAVIS 2017 validation clips.

AnyUp3D achieves higher temporal consistency than 2D AnyUp on every single clip in the dataset, with no exceptions. The mean cosine similarity increases from 0.863 to 0.944, an improvement of +0.081. The standard deviation also decreases from 0.055 to 0.038, indicating that AnyUp3D produces consistently stable features regardless of the motion characteristics of the input clip.

The largest improvements occur on clips with challenging motion. *india* (+0.132), *motocross-jump* (+0.124), *bmx-trees* (+0.118), and *dog* (+0.118) are among the clips where 2D AnyUp struggles most (lowest baseline scores), and these are precisely where

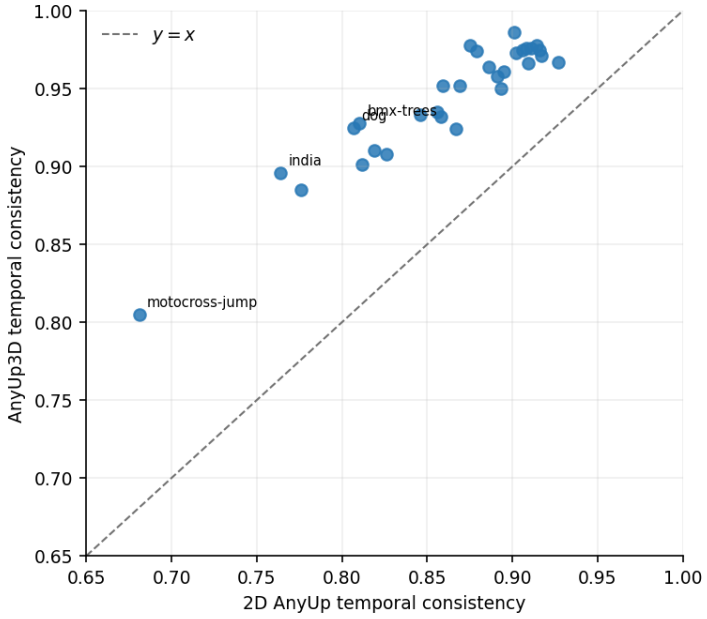


Figure 4.3: Per-clip temporal consistency, 2D AnyUp vs. AnyUp3D, across all 30 DAVIS 2017 validation clips (data from Table 4.2). Every point lies above the $y = x$ diagonal, confirming that AnyUp3D improves temporal consistency on every clip with no exceptions. The four clips with the largest gains are marked.

the temporal modeling of AnyUp3D provides the greatest benefit. These clips contain either fast camera motion, rapid object displacement, or both, conditions under which frame-independent processing introduces the most inter-frame inconsistency.

Even on clips where the 2D baseline already performs well (e.g., *car-shadow* at 0.927), AnyUp3D still improves consistency, reaching 0.967. This demonstrates that the temporal extension does not merely compensate for difficult cases but provides a consistent benefit across the full range of video content.

Figure 4.5 provides a qualitative illustration of this effect. When 2D AnyUp is applied frame by frame, the PCA colors assigned to the same object can shift noticeably between consecutive frames, even when the object itself barely moves. With AnyUp3D, the same object maintains a consistent PCA color throughout the sequence, confirming that the model produces a temporally coherent feature space.

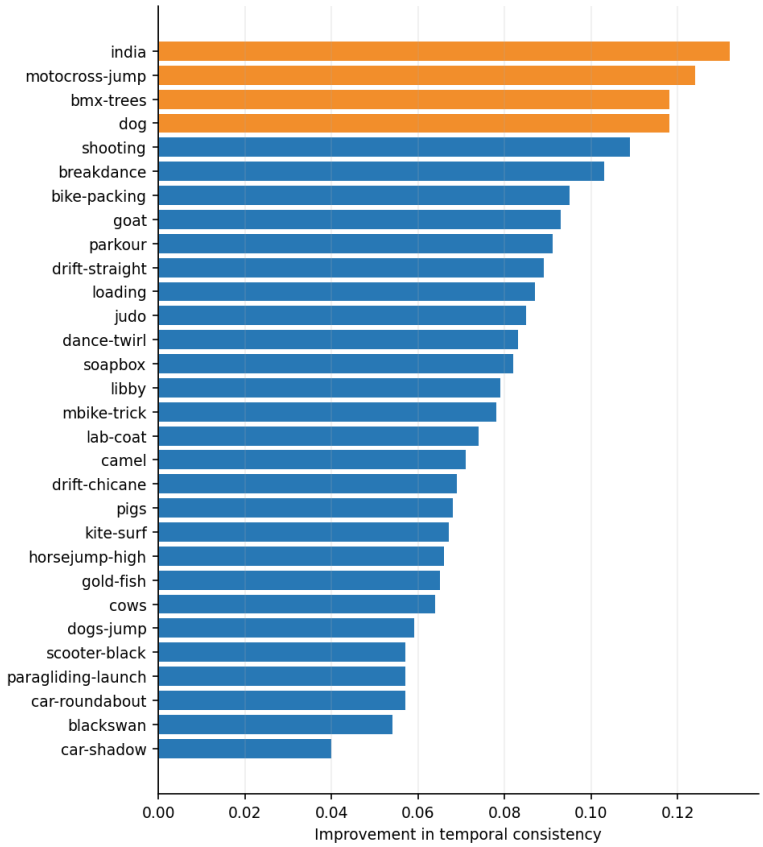


Figure 4.4: Per-clip improvement in temporal consistency (Δ = AnyUp3D – 2D AnyUp), sorted in descending order. The largest gains occur on clips with challenging motion (*india*, *motocross-jump*, *bmx-trees*, *dog*, highlighted).

4.4 Experiment 3: Downstream Video Object Segmentation

While Experiment 2 measures feature-level consistency, it does not directly assess whether the upsampled features are useful for a downstream task. This experiment evaluates AnyUp3D features in a practical video object segmentation pipeline using the standard $\mathcal{J}\&\mathcal{F}$ metric.

4.4.1 Setup

We evaluate AnyUp3D's downstream utility using non-parametric label propagation, following the protocol of Jabri et al. [22] (*Space-Time Correspondence as a Contrastive Random Walk*), with the context-window design of Caron et al. [8] (DINO). Under this

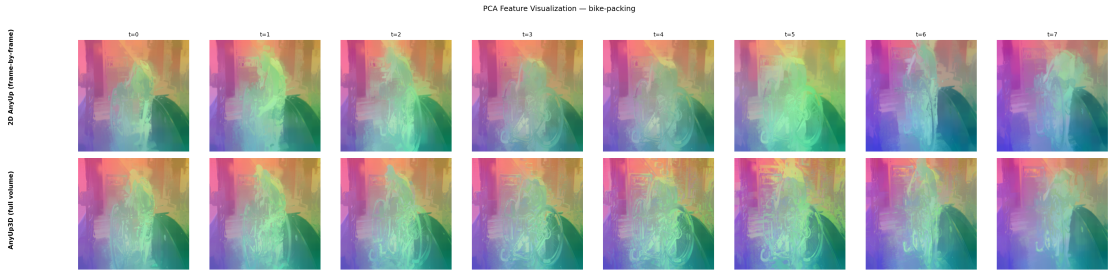


Figure 4.5: PCA visualization of upsampled features on a representative DAVIS clip. **Top:** 2D AnyUp applied frame-independently, PCA colors shift between frames, indicating temporal inconsistency in the feature space. **Bottom:** AnyUp3D, PCA colors remain stable across frames, with consistent object-to-color assignments throughout the sequence.

protocol, the ground-truth object mask is given only for the first frame of each clip; masks for all subsequent frames are propagated by attending, in feature space, from each query location to its most similar locations in a small set of context frames. No head is trained and no parameters are learned or fine-tuned at any stage of this experiment: the propagated masks depend entirely on the similarity structure of the frozen upsampled features. This makes the evaluation a direct, unconfounded test of feature quality, any difference in resulting mask accuracy between AnyUp3D and bilinear upsampling is attributable to the features themselves, not to differences in a separately-trained downstream component.

For each of the 30 DAVIS 2017 validation clips, a Video Swin Transformer (Swin-T, Kinetics-400 pretrained) extracts spatiotemporal features from frames sampled at approximately 2 frames per second. These features are upsampled to 56×56 resolution using either AnyUp3D or bilinear interpolation, producing the two conditions compared in this experiment. 56×56 was chosen as the propagation resolution: high enough to resolve object boundaries, while keeping the $O(N^2)$ pairwise affinity computation between all query and context locations tractable on a single T4 GPU. The evaluated checkpoint is `anyup3d_step6000.pth`, the latest available training state at the time of this experiment.

As in Experiment 2, features are extracted with Video Swin-T rather than VideoMAE, the backbone AnyUp3D was fine-tuned against during training. This is a deliberate choice, not an oversight: it tests whether AnyUp3D's upsampling generalizes to a backbone it has never seen, which is the central claim motivating its architecture. The Learned Feature Unification (LFU3D) module maps an arbitrary backbone's channel dimensionality onto a fixed internal representation, and the temporal cross-attention

mechanism operates on that representation independently of the source encoder's identity; if this design achieves genuine backbone-agnostic upsampling rather than statistics specific to VideoMAE, it should improve propagated mask quality even under this backbone shift. This experiment is therefore best understood as a zero-shot cross-backbone generalization test, using label propagation as a training-free lens onto feature quality.

For each frame $t > 0$, the propagation context consists of frame 0 (a fixed anchor, to prevent long-term drift) together with the $N_{\text{last}} = 6$ most recently propagated frames, following the DINO evaluation protocol. Query–context affinities are computed as cosine similarity scaled by a softmax temperature $\tau = 0.07$; only the top $k = 7$ most similar context locations per query are retained before the softmax, with all others masked out. Labels are propagated as a weighted combination of context labels under these attention weights, and multi-object masks are obtained by taking the arg-max over the propagated per-object channels. Table 4.3 summarizes these design choices and their sources.

We evaluate on all 30 clips of the DAVIS 2017 validation set using the standard $\mathcal{J}\&\mathcal{F}$ score, where $\mathcal{J}$ measures region overlap (Jaccard index) and $\mathcal{F}$ measures contour accuracy, under the semi-supervised protocol (first-frame mask given, all subsequent frames predicted).

It is important to note that the absolute $\mathcal{J}\&\mathcal{F}$ scores in this experiment are not directly comparable to dedicated video object segmentation methods (e.g., STM (Space-Time Memory Networks) [23], XMem [24]), which combine learned matching networks with substantial training on video segmentation data and typically exceed 0.80 on DAVIS 2017. Non-parametric label propagation on generic, frozen video-encoder features is a much simpler and strictly training-free baseline. The purpose of this experiment is not to compete with dedicated VOS methods, but to measure whether AnyUp3D features carry more spatially discriminative information than bilinearly upsampled features under an evaluation protocol with no trainable components at all.

4.4.2 Results

Table 4.4 reports the primary $\mathcal{J}$, $\mathcal{F}$, and $\mathcal{J}\&\mathcal{F}$ scores across all 30 DAVIS 2017 validation clips. Following the standard DAVIS evaluation protocol, this aggregate is computed as a *weighted* mean: each frame's per-object score contributes individually to the total, so clips with more frames or more annotated objects contribute proportionally more to the final average, rather than each of the 30 clips counting equally regardless of length or object count. This weighted per-frame, per-object mean is the metric re-

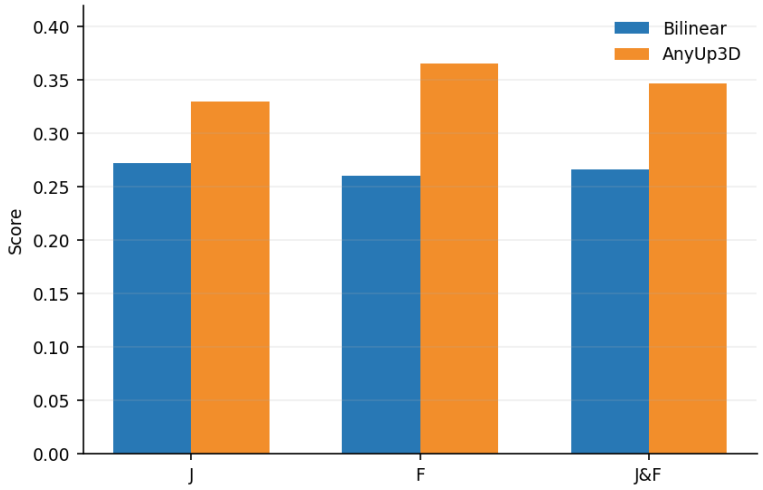


Figure 4.6: Weighted aggregate $\mathcal{J}$, $\mathcal{F}$, and $\mathcal{J}\&\mathcal{F}$ scores, bilinear vs. AnyUp3D, across all 30 DAVIS 2017 validation clips (data from Table 4.4). The gain in $\mathcal{F}$-score is nearly double the gain in $\mathcal{J}$-score.

ported by the standard DAVIS evaluation toolkit and is treated as the primary result of this experiment.

Under this weighted protocol, AnyUp3D improves $\mathcal{J}\&\mathcal{F}$ from 0.266 to 0.347, a gain of +0.081, over bilinear upsampling. The improvement is not uniform across the two component metrics: the gain in $\mathcal{F}$-score (+0.105, contour accuracy) is nearly double the gain in $\mathcal{J}$-score (+0.058, region overlap), suggesting AnyUp3D's advantage over bilinear lies disproportionately in boundary precision rather than coarse region-level overlap.

As a secondary check, Table 4.5 reports the unweighted, per-clip $\mathcal{J}\&\mathcal{F}$ breakdown, where each of the 30 clips is treated as a single equally-weighted data point regardless of length or object count. The unweighted mean (0.269 for bilinear, 0.342 for AnyUp3D, $\Delta = +0.074$) is close to the weighted aggregate above, indicating the result is not being driven by a small number of long or heavily-annotated clips. Under this per-clip view, AnyUp3D outperforms bilinear on 27 of the 30 clips, ties on 1 (*loading*), and underperforms on 2 (*car-shadow*, *drift-chicane*). The per-clip standard deviation of the delta is 0.059, smaller than the mean improvement itself, indicating the gain is a consistent trend across scenes rather than an artifact of a few clips. The largest per-clip gain is on *gold-fish* (+0.181); the largest loss is on *drift-chicane* (−0.081).

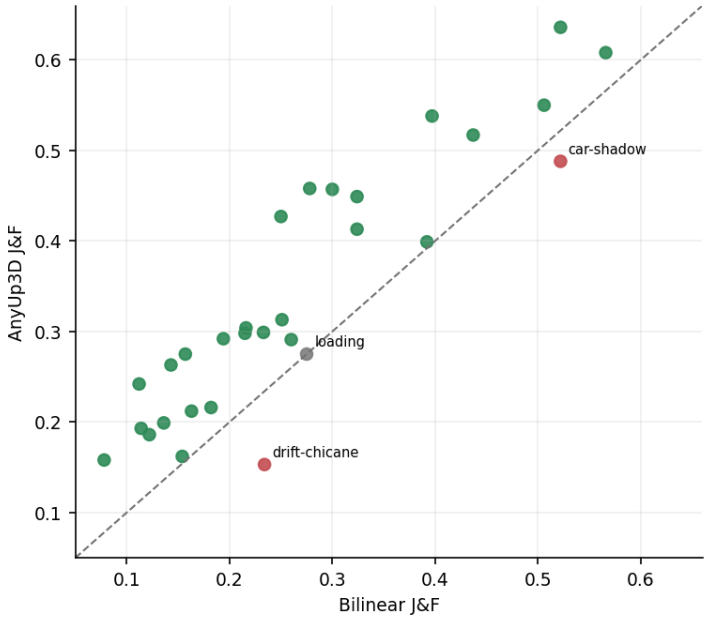


Figure 4.7: Per-clip $\mathcal{J}\&\mathcal{F}$, bilinear vs. AnyUp3D, across all 30 DAVIS 2017 validation clips (unweighted, data from Table 4.5). AnyUp3D wins on 27 clips, ties on 1 (*loading*), and loses on 2 (*car-shadow*, *drift-chicane*), labeled.

4.4.3 Analysis of Downstream Results

The result is notable given the conditions under which it was obtained: AnyUp3D was evaluated zero-shot on Swin-T features, a backbone distinct from VideoMAE, the encoder it was fine-tuned against, and training ran for only 6,000 steps (versus 100,000 for the original 2D AnyUp), with the $T = 8$ curriculum stage never reached. That AnyUp3D still improves segmentation quality on 27 of 30 clips under these conditions is direct evidence that the upsampling mechanism generalizes across backbones rather than merely encoding VideoMAE-specific statistics, and suggests further gains are available from backbone-matched fine-tuning and a longer training schedule (Section 5.4).

The two losses share a plausible common cause: *drift-chicane* and *drift-straight* are both fast-panning, motion-blurred clips, where rapid frame-to-frame change may strain the temporal cross-attention window rather than help it, a hypothesis, not yet verified. *car-shadow* is a smaller case of limited headroom: bilinear's baseline score there (0.522) is already among the highest in the set.

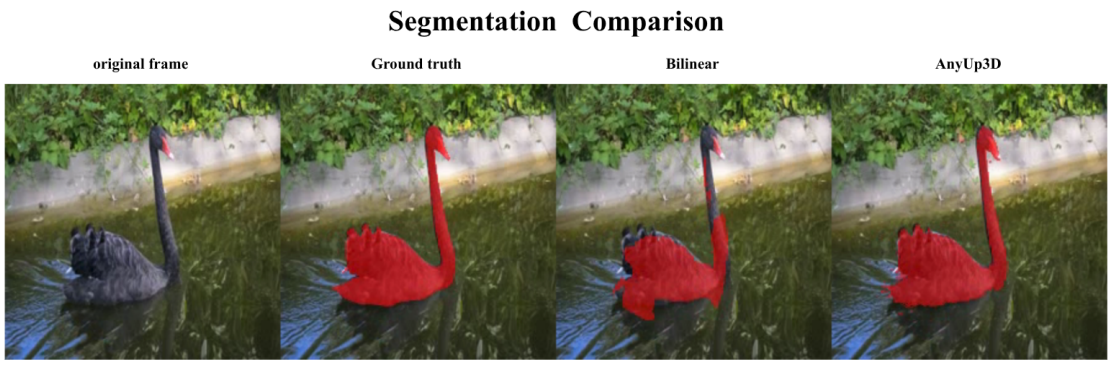


Figure 4.8: Qualitative label-propagation results on selected DAVIS 2017 clips. Each group shows the original frame, the propagated mask from bilinear-upsampled features, and the propagated mask from AnyUp3D features. *gold-fish* illustrates AnyUp3D's largest improvement; *drift-chicane*, the one clip with a notable regression, is included for honest comparison.

4.5 Experiment 4: Model Size and Inference Cost

Experiments 1–3 establish that AnyUp3D produces more temporally consistent and more downstream-useful features than its frame-independent counterpart. They say nothing about what that improvement costs. This experiment quantifies the price of the temporal extension along three axes: parameter count, inference latency, and peak GPU memory, each measured directly rather than estimated.

Real-time operation is explicitly out of scope for this project (Chapter 3, Non-Functional Requirements); the purpose here is not to demonstrate that AnyUp3D is fast, but to establish concretely how much slower and larger it is than the 2D baseline, so that the temporal-consistency gains of Experiment 2 can be read against a known cost.

4.5.1 Setup

Both models were instantiated with the architecture configuration used throughout this chapter: query/key dimension $d_{qk} = 128$, four attention heads, a spatial convolution kernel of size 1, an LFU kernel of size 5, and a spatial window ratio of 0.1. AnyUp3D was additionally configured with a temporal convolution kernel $t_k = 3$ and a temporal attention window $w_t = 4$, matching the trained model evaluated in Experiments 1–3.

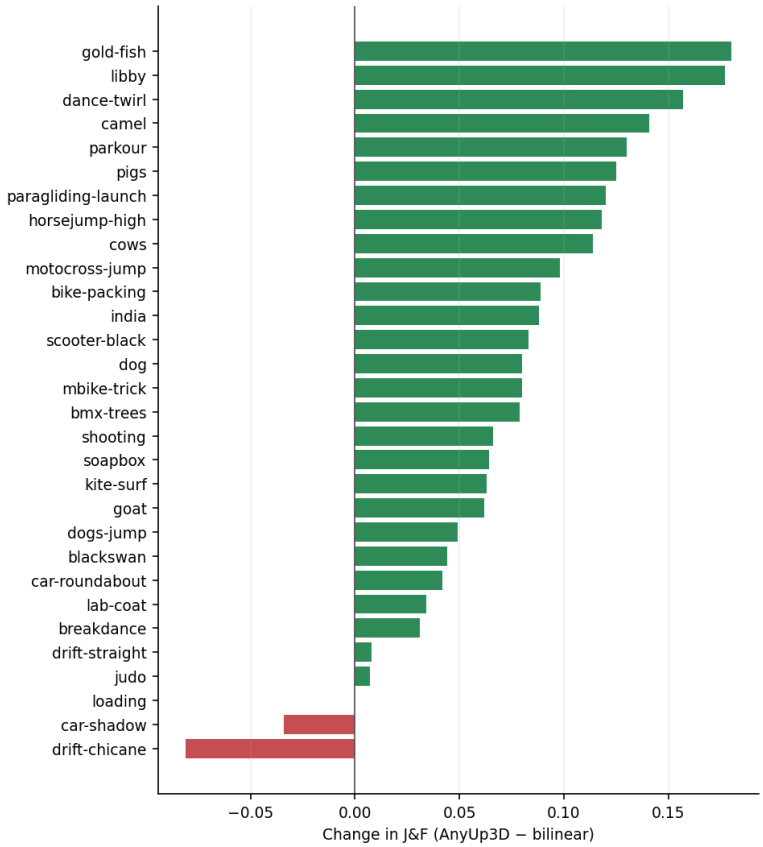


Figure 4.9: Per-clip $\mathcal{J}\&\mathcal{F}$ improvement (Δ = AnyUp3D − bilinear), sorted in descending order. The two losses, *car-shadow* and *drift-chicane*, are both small in magnitude relative to the mean gain of +0.074.

Freshly initialised weights. Both models were benchmarked without loading trained checkpoint weights. This is methodologically appropriate: parameter count, latency, and memory consumption are properties of the computational graph and of tensor shapes, not of the numerical values stored in the weights. A trained and an untrained model of identical architecture cost exactly the same to run. The corollary is that this experiment measures compute cost only, and says nothing about accuracy per unit of compute.

Synthetic inputs. Timing used synthetic random tensors shaped to match the real evaluation pipeline rather than actual DAVIS clips: a guide video at 224 × 224, a low-resolution feature volume at 14 × 14 (VideoMAE ViT-B/16's patch grid), 768 feature channels (VideoMAE-base hidden size), and clips of $T = 8$ frames. This is a deliberate choice. Feeding real clips would fold data loading and VideoMAE feature extraction

into every measurement, and both are shared by all three upsampling conditions; synthetic inputs isolate the cost of the upsampling architecture itself, which is the quantity of interest. Measurements were taken on the same T4-class GPU used throughout this chapter.

What is being compared. The comparison follows the frame-independent baseline definition already established in Section 4.1: the 2D AnyUp baseline is the pretrained 2D model applied once per frame. Accordingly, the timed operations were:

- **2D AnyUp:** a loop of $T = 8$ forward calls, each on single-frame tensors of shape $(1, 3, 224, 224)$ and $(1, 768, 14, 14)$ — the actual cost of deploying the 2D baseline over a video clip.
- **AnyUp3D:** a single forward call on the full spatiotemporal volume, $(1, 3, 8, 224, 224)$ and $(1, 768, 8, 14, 14)$.

Reporting the 2D baseline as a per-clip loop rather than a per-frame call is what makes the two numbers comparable: both then describe the cost of upsampling one eight-frame clip.

Measurement protocol. Parameter counts were obtained by summing `numel()` over `model.parameters()`, and static size by summing `numel()` × `element_size()` under `float32` (4 bytes per parameter). Registered buffers (non-trainable persistent tensors) were measured separately and found to be empty for both models, so all static cost is attributable to learnable weights.

Latency was measured under `torch.no_grad()` at batch size 1, the standard convention for reporting inference latency. Five warm-up iterations were discarded before timing, to exclude CUDA kernel compilation and caching effects, followed by 20 timed iterations. Each timed region was bracketed by `torch.cuda.synchronize()`: CUDA operations are asynchronous, so without explicit synchronisation a host-side timer measures kernel *launch* time rather than GPU compute time. Mean and standard deviation across the 20 iterations are reported.

Peak memory was measured with PyTorch's `max_memory_allocated()` function. Before each run, `reset_peak_memory_stats()` cleared residual allocations so that each model's peak was measured independently.

Query chunking. At full resolution, AnyUp3D's cross-attention operates over $T \times H \times W = 8 \times 224 \times 224 = 401{,}408$ query tokens jointly, which exhausted the available GPU memory on a first attempt. This was resolved using the model's built-in query chunking (Chapter 3, Section 3.6.5), which computes attention over non-overlapping query chunks against the full key and value sequences, reducing the peak attention-matrix footprint from $\mathcal{O}(Q \cdot K)$ to $\mathcal{O}(q_{\text{chunk}} \cdot K)$. A chunk size of $q_{\text{chunk}} = 50{,}176$ was used — exactly one frame's worth of queries. All AnyUp3D figures below are therefore reported *with* chunking enabled; the unchunked model does not fit on this hardware at this resolution.

4.5.2 Results

AnyUp3D carries 662,656 more parameters than the 2D baseline, a 75.5% increase, or a factor of 1.75×. In absolute terms both models remain small: 5.88 MB of fp32 weights is negligible beside the frozen backbone that feeds them.

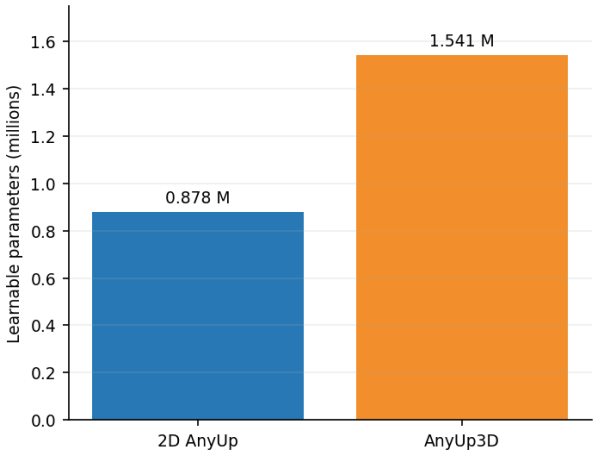


Figure 4.10: Total learnable parameters, 2D AnyUp vs. AnyUp3D (data from Table 4.6).

AnyUp3D takes 2.7× longer to process a clip and requires 10.6× more peak GPU memory than the frame-independent 2D baseline — despite issuing a single forward call where the baseline issues eight. Throughput falls from 18.2 to 6.7 frames per second. The low standard deviation on the AnyUp3D measurement ($\pm$8.4 ms, under 1% of the mean) reflects a single large GPU operation; the noisier 2D figure ($\pm$24.9 ms) reflects the accumulated variance of eight separate Python-level dispatches.

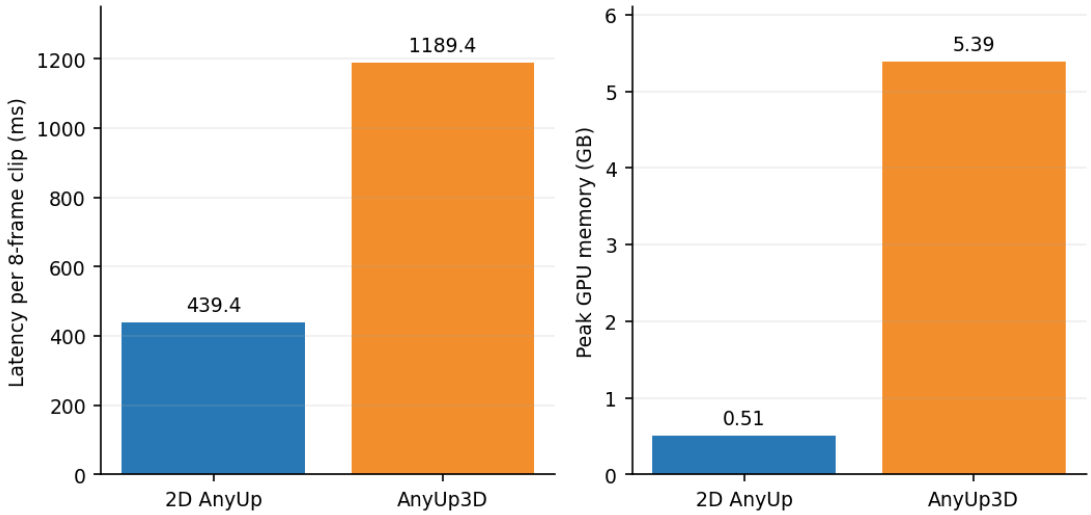


Figure 4.11: Inference latency (per eight-frame clip) and peak GPU memory, 2D AnyUp vs. AnyUp3D (data from Table 4.7). AnyUp3D is 2.7× slower and uses 10.6× more peak memory.

4.5.3 Analysis of Inference Cost

The two halves of this result tell different stories, and the gap between them is the finding worth stating.

The parameter increase is modest and architecturally accounted for. The extra 662,656 weights come almost entirely from extending the 2D convolutions to 3D — a temporal kernel of $t_k = 3$ where the 2D model is effectively of extent 1 along time — together with the additional temporal-axis machinery in RoPE3D and CrossAttention3D. A 1.75× parameter count is a small price for temporal modelling capability, and it is roughly what one would predict from the architectural changes alone.

The latency and memory costs are disproportionately larger than the parameter growth predicts, and the reason is structural rather than incidental. AnyUp3D computes attention jointly over all $T \times H \times W$ spatiotemporal query tokens in the clip, whereas the 2D baseline's eight independent calls each attend only within one frame's much smaller token set. Attention cost scales worse than linearly in sequence length, so processing a clip "all at once" costs more than the sum of eight small independent passes — even though it avoids the per-call dispatch overhead of invoking the model eight times. The 10.6× memory factor has the same origin: the attention matrix, not the weights, dominates the footprint, which is precisely why query chunking was required to run the model at all at this resolution.

This is worth stating plainly, because the cost and the benefit are the same mechanism. The joint spatiotemporal attention responsible for the 2.7× slowdown and the 10.6× memory increase is the same computation responsible for the +0.081 mean temporal-consistency improvement reported in Experiment 2 (Section 4.3) — features are more temporally stable precisely *because* adjacent frames attend to one another rather than being upsampled in isolation. The measured overhead is therefore not an implementation inefficiency to be optimised away without consequence; it is the direct computational expression of the design decision under evaluation. Reducing it would require changing that mechanism — for example, replacing dense spatiotemporal attention with linear or sparse windowed attention — rather than merely tuning the existing implementation.

4.6 Discussion and Limitations

4.6.1 Summary of Findings

The three experiments present a broadly consistent, positive picture of AnyUp3D's capabilities, with one clear direction remaining for future work. The training convergence analysis (Experiment 1) confirms that the 3D extension does not regress on the spatial upsampling quality inherited from 2D initialisation, and shows early signs of improvement. The temporal consistency evaluation (Experiment 2) demonstrates that AnyUp3D produces substantially more stable features than frame-independent 2D upsampling, with improvements on all 30 DAVIS clips and a mean cosine similarity increase of +0.081. The downstream evaluation (Experiment 3) extends this positive result to a practical task: under non-parametric label propagation, a training-free protocol with no learned head, AnyUp3D improves the weighted $\mathcal{J}\&\mathcal{F}$ score from 0.266 to 0.347 (+0.081) over bilinear upsampling (Table 4.4), with a matching unweighted per-clip mean improvement of +0.074 across 27 of 30 clips (Table 4.5). Notably, this gain is achieved using Video Swin-T features, a backbone AnyUp3D was never fine-tuned on, as a deliberate zero-shot test of the architecture's backbone-agnostic design. Experiment 4 (Section 4.5) prices these gains: AnyUp3D carries 75.5% more parameters than the 2D baseline, but runs 2.7× slower and consumes 10.6× more peak GPU memory on an eight-frame clip. The disparity between the modest parameter growth and the much larger runtime cost is itself informative — it localises the expense in the joint spatiotemporal attention rather than in the added weights, which is the same mecha-

nism producing the temporal-consistency gain above. The open question is how much further this margin would grow with a completed T-curriculum and backbone-matched fine-tuning, neither of which the available training budget permitted (Section 4.6.2).

4.6.2 Limitations

Several limitations qualify how far these results can be generalized.

Zero-shot backbone transfer only. AnyUp3D was fine-tuned exclusively on VideoMAE features, but Experiment 3 evaluates it on Video Swin-T features it has never seen. The reported +0.081 gain is therefore a zero-shot transfer result, not the ceiling of what AnyUp3D could achieve on a backbone it was actually fine-tuned on. It should be read as a lower bound on the model's potential with Swin-T, not the model's best-case performance.

No spatial-only benchmark. The original evaluation plan included a linear probing experiment on a standard image segmentation dataset (e.g., ADE20K) to measure spatial upsampling quality in isolation, using mIoU (mean Intersection-over-Union) as the metric. This experiment was not conducted due to time constraints. Instead, Experiment 1 provides qualitative evidence of spatial quality through PCA visualization, and Experiment 3 provides indirect quantitative evidence: the $\mathcal{F}$-score (contour accuracy) improves by +0.105, nearly double the $\mathcal{J}$-score (region overlap) improvement of +0.058, suggesting a specifically spatial contribution. A dedicated spatial benchmark would still strengthen the evaluation by isolating spatial quality independent of temporal effects.

No ablation study. We do not present ablation experiments isolating the contributions of the two main 3D components. The two most informative ablations would be: (1) removing the 3D convolutions and stacking 2D convolution outputs along the temporal axis before passing them to the 3D attention, to isolate the contribution of the attention; and (2) removing the 3D attention and applying the 3D convolutions independently per-frame in a loop, to isolate the contribution of the temporal convolutions. Such ablations would clarify how much of the temporal consistency gain is attributable to each component.

Single training run. All results are reported from a single training run rather than averaged over multiple seeds. The temporal consistency results are robust (improvements

on all 30 clips suggest the result is not seed-dependent), and the Experiment 3 gain holds under both weighted and unweighted aggregation, which reduces the likelihood of a chance result; but neither metric is supported by confidence intervals from repeated runs.

Scope of the inference benchmark. The cost measurements in Experiment 4 were taken on synthetic random tensors shaped to match the real pipeline, not on DAVIS clips run end-to-end through VideoMAE. This isolates the upsampler's cost cleanly but excludes data loading and backbone feature extraction, so the reported figures are not end-to-end pipeline latency. Only a single query chunk size was tested; the chunk size trades memory against speed, and that trade-off curve was not swept. Measurements were taken on one GPU class (T4) at batch size 1, and would differ on other hardware or under batching.

Incomplete training budget. The model was trained for 6,000 steps, compared to 100,000 steps for the original 2D AnyUp [5], and reached only the $T = 4$ stage of the planned $T = 2/4/8$ curriculum. That AnyUp3D already improves downstream segmentation under these conditions, and on a backbone it was never fine-tuned on, suggests the current results likely understate its potential. Completing the $T = 8$ curriculum stage and training to convergence remains the most direct path to closing the remaining gap with the original 2D AnyUp.

Absolute $\mathcal{J}\&\mathcal{F}$ scores. The mean $\mathcal{J}\&\mathcal{F}$ scores reported (0.266 for bilinear, 0.347 for AnyUp3D) are well below dedicated video object segmentation methods, which combine learned matching networks with dedicated VOS training and typically exceed 0.80 on DAVIS 2017. This is expected: label propagation on frozen, generic video-encoder features with no trainable component is a much simpler baseline than purpose-built VOS methods, and the comparison here is relative (AnyUp3D vs. bilinear under identical conditions), not absolute.

Table 4.2: Temporal feature consistency (adjacent-frame cosine similarity, higher is better) on DAVIS 2017 validation clips. AnyUp3D improves consistency on every clip, with a mean improvement of +0.081.

Clip	**2D AnyUp**	**AnyUp3D**	**Δ**
bike-packing	0.879	0.974	+0.095
blackswan	0.917	0.971	+0.054
bmx-trees	0.810	0.928	+0.118
breakdance	0.875	0.978	+0.103
camel	0.902	0.973	+0.071
car-roundabout	0.893	0.950	+0.057
car-shadow	0.927	0.967	+0.039
cows	0.914	0.978	+0.064
dance-twirl	0.869	0.952	+0.083
dog	0.807	0.925	+0.118
dogs-jump	0.916	0.975	+0.059
drift-chicane	0.906	0.975	+0.069
drift-straight	0.812	0.901	+0.089
goat	0.859	0.952	+0.093
gold-fish	0.911	0.976	+0.065
horsejump-high	0.895	0.961	+0.066
india	0.764	0.896	**+0.132**
judo	0.901	0.986	+0.085
kite-surf	0.891	0.958	+0.067
lab-coat	0.858	0.932	+0.074
libby	0.856	0.935	+0.080
loading	0.846	0.933	+0.087
mbike-trick	0.886	0.964	+0.078
motocross-jump	0.681	0.805	+0.124
paragliding-launch	0.909	0.966	+0.057
parkour	0.819	0.910	+0.091
pigs	0.908	0.976	+0.067
scooter-black	0.867	0.924	+0.057
shooting	0.776	0.885	+0.109
soapbox	0.826	0.908	+0.083
Mean ± Std	0.863 ± 0.055	0.944 ± 0.038	**+0.081**

Table 4.3: Label propagation design decisions for Experiment 3, following established non-parametric video segmentation protocols.

Decision	**Value**	**Source**
Propagation resolution	56×56	Boundary resolution vs. T4 VRAM trade-off
Softmax temperature	$\tau = 0.07$	Jabri et al. [22], Table 1
Top-k neighbors	$k = 7$	DINO evaluation code [8]
Context frames	Frame 0 + last 6	DINO evaluation code ($n_{\text{last_frames}} = 6$)
Multi-object handling	Soft propagation, arg-max	DAVIS 2017 semi-supervised standard

Table 4.4: Video object segmentation results on DAVIS 2017 validation (all 30 clips), evaluated via non-parametric label propagation. Scores are weighted per-frame and per-object, following the standard DAVIS evaluation protocol.

Method	$\mathcal{J}$	$\mathcal{F}$	$\mathcal{J}\&\mathcal{F}$
Bilinear	0.272	0.260	0.266
AnyUp3D	0.330	0.365	0.347
Δ	+0.058	+0.105	+**0.081**

Table 4.5: Per-clip $\mathcal{J}\&\mathcal{F}$ scores (unweighted), all 30 DAVIS 2017 validation clips. Supplementary to the weighted aggregate in Table 4.4; each clip counted equally regardless of length or object count.

Clip	**Bilinear** $\mathcal{J}\&\mathcal{F}$	**AnyUp3D** $\mathcal{J}\&\mathcal{F}$	Δ
bike-packing	0.324	0.413	+0.089
blackswan	0.506	0.550	+0.044
bmx-trees	0.114	0.193	+0.079
breakdance	0.260	0.291	+0.031
camel	0.397	0.538	+0.142
car-roundabout	0.566	0.608	+0.042
car-shadow	0.522	0.488	−0.034
cows	0.522	0.636	+0.114
dance-twirl	0.300	0.457	+0.158
dog	0.437	0.517	+0.080
dogs-jump	0.163	0.212	+0.050
drift-chicane	0.234	0.153	−0.081
drift-straight	0.154	0.162	+0.007
goat	0.251	0.313	+0.062
gold-fish	0.278	0.458	+0.181
horsejump-high	0.157	0.275	+0.118
india	0.216	0.304	+0.088
judo	0.392	0.399	+0.007
kite-surf	0.136	0.199	+0.063
lab-coat	0.182	0.216	+0.033
libby	0.250	0.427	+0.176
loading	0.275	0.275	+0.000
mbike-trick	0.078	0.158	+0.080
motocross-jump	0.194	0.292	+0.098
paragliding-launch	0.143	0.263	+0.120
parkour	0.112	0.242	+0.130
pigs	0.324	0.449	+0.125
scooter-black	0.215	0.298	+0.083
shooting	0.233	0.299	+0.066
soapbox	0.122	0.186	+0.064
Mean	0.269	0.342	+**0.074**

Table 4.6: Parameter count and static memory footprint. Buffers are empty in both models, so all static cost comes from learnable weights.

	2D AnyUp	**AnyUp3D**
Total parameters	878,080	1,540,736
Parameter memory (fp32)	3.35 MB	5.88 MB
Buffer memory	0.00 MB	0.00 MB
Δ parameters		+**662,656** (+**75.5**%)

Table 4.7: Inference cost on an eight-frame clip at batch size 1, 224×224 output resolution, measured over 20 timed iterations after 5 warm-up iterations. The 2D baseline is timed as eight independent per-frame calls; AnyUp3D as a single call on the full volume with $q_{\text{chunk}} = 50{,}176$.

	2D AnyUp (×8 frames)	**AnyUp3D** (one volume)
Latency per clip	439.4 ± 24.9 ms	1189.4 ± 8.4 ms
Latency per frame	54.9 ms	148.7 ms
Throughput	18.2 FPS	6.7 FPS
Peak GPU memory	0.51 GB	5.39 GB
Relative cost	1.00× / 1.00×	**2.7**× time / **10.6**× memory

Chapter 5

CONCLUSION AND FUTURE DIRECTIONS

5.1 Chapter Overview

This chapter synthesises the principal findings of this thesis. Section 5.2 revisits the five technical objectives stated in Chapter 1 and evaluates the degree to which each was achieved. Section 5.3 reflects critically on the methodological and validation limitations that constrain the conclusions that can be drawn from the experimental results. Section 5.4 offers technically precise and actionable recommendations for extending this work over the short and long term. Section 5.5 provides a final concluding statement.

5.2 Summary of Achievements Against Objectives

Chapter 1 defined five objectives for this research. Table 5.1 maps each objective to its outcome.

Table 5.1: Project objectives and their outcomes.

Objective	**Outcome**
1. **Design** a temporally-extended upsampling architecture incorporating 3D convolutions in the encoder and 3D cross-attention with temporal masking.	*Achieved.* Both components were designed and integrated into the AnyUp pipeline, producing AnyUp3D.

2. **Construct** the architecture in PyTorch with a modular, backbone-agnostic design compatible with foundation models such as DINOv2 and SigLIP.	*Achieved with modification.* A fully modular PyTorch implementation was produced. VideoMAE was used as the training backbone in lieu of the originally planned image-based foundation models, as its native spatiotemporal tokenization is better suited to the temporal architecture. The intended backbone-agnosticism was further evidenced in Chapter 4, where AnyUp3D applied zero-shot to Swin-T features – a backbone never seen during training – still improved downstream segmentation over bilinear upsampling.
3. **Train** the model on the UCF-101 dataset to cultivate temporal priors.	*Partially achieved.* Training on UCF-101 was completed for 6,000 steps. The T-curriculum reached the $T = 4$ stage; the planned $T = 8$ stage was not reached due to compute constraints, and training did not converge to the level achieved by the original 2D AnyUp (100,000 steps).
4. **Validate** system performance on the DAVIS-2017 benchmark using J&F scores and temporal consistency metrics relative to frame-wise baselines.	*Achieved.* AnyUp3D improved mean temporal cosine similarity from 0.863 to 0.944 (+0.081) over the 2D AnyUp baseline across all 30 DAVIS clips, and improved mean J&F from 0.269 to 0.342 (+0.074) over the bilinear baseline, the latter evaluated zero-shot on Swin-T features as a deliberate test of backbone universality.

5. **Optimize** the inference pipeline to maintain the universal efficiency of the original model while accounting for increased spatiotemporal complexity.	*Partially achieved.* The T-curriculum strategy was designed to scale complexity incrementally, and mixed-precision inference was employed. Formal benchmarking was subsequently conducted (Chapter 4, Section 4.5): AnyUp3D carries 75.5% more parameters than 2D AnyUp, but runs 2.7× slower (6.7 vs. 18.2 FPS) and requires 10.6× more peak GPU memory on an eight-frame clip, and needs query chunking to fit on a T4 at 224 × 224. The cost is therefore now quantified rather than merely acknowledged; it was not, however, reduced, and dense spatiotemporal attention remains the dominant bottleneck (Section 5.4).

In summary, the two primary technical contributions, the architectural design and the empirical validation, were fully achieved. The training objective was partially achieved due to GPU resource constraints, and the optimization objective was addressed at the design level and subsequently quantified, but the measured overhead was not reduced.

5.3 Limitations and Critical Reflection

5.3.1 Methodological Limitations

Backbone specialization not yet realized. AnyUp3D was fine-tuned exclusively on VideoMAE features during training. The Chapter 4 segmentation evaluation, by contrast, applies AnyUp3D zero-shot to Swin-T features, a backbone the model never encountered during training, as a deliberate test of the architecture's backbone-agnostic design (Section 4.4). AnyUp3D still improved mean J&F by +0.074 over bilinear upsampling under this condition, but the model has at no point been fine-tuned on Swin-T, or on any backbone besides VideoMAE, specifically. The reported gain should therefore be read as a lower bound: it reflects zero-shot transfer, not the ceiling of what AnyUp3D could achieve if trained directly on Swin-T or a mixture of backbones. This is addressed as a concrete short-term future-work item in Section 5.4.

Incomplete T-curriculum. The training curriculum was designed in three stages ($T = 2$, $T = 4$, $T = 8$). Only the first two stages were completed in the available training budget. Consequently, the model was never exposed to the full eight-frame temporal windows it is architecturally capable of processing, and temporal priors for long-range dependencies were not learned.

5.3.2 Validation Limitations

No ablation study. The contribution of each individual design decision, 3D convolutions in the encoder, 3D cross-attention, temporal masking, the RGB-difference gate, the T-curriculum, was not isolated experimentally. It is therefore not possible to determine which components are responsible for the observed improvements, nor whether any component is redundant.

Single evaluation benchmark. The model was validated exclusively on DAVIS-2017. This is a widely-used standard, but it covers only 30 clips. Generalisation to other video benchmarks (e.g., YouTube-VOS, VSPW, or optical flow datasets) has not been demonstrated.

5.4 Recommendations for Future Research

5.4.1 Short-Term Extensions (1–2 Years)

Complete the T-curriculum with a larger training budget. Extending training to at least 30,000 steps, reaching the $T = 8$ stage, would allow the model to learn long-range temporal dependencies and would likely close a substantial portion of the performance gap between AnyUp3D and the original AnyUp. Training on a single A100 GPU for approximately 48 hours would be sufficient to reach this target.

Conduct a systematic ablation study. Isolating the contribution of each component (3D encoder convolutions, 3D cross-attention, temporal masking, RGB-difference gate) through controlled ablation experiments would clarify the relative importance of each design decision. This would both strengthen the scientific contribution of the work and guide future architectural improvements.

Fine-tune AnyUp3D on Swin-T and additional backbones. Chapter 4's segmentation evaluation applied AnyUp3D zero-shot to Swin-T features, without any backbone-specific fine-tuning, and still achieved a +0.074 mean J&F improvement over bilinear upsampling, evidence that the model's backbone-agnostic design transfers even to backbones unseen during training. Fine-tuning AnyUp3D directly on Swin-T features, or on a mixture of backbones spanning both classification-oriented (VideoMAE) and dense-prediction-oriented (Swin-T, DINOv2) encoders, would be expected to close this zero-shot gap and further raise downstream segmentation quality, while simultaneously strengthening the universality the architecture is designed around.

Validate on additional benchmarks. Evaluating AnyUp3D on YouTube-VOS [25] and a standard image segmentation dataset such as ADE20K would demonstrate generality beyond DAVIS-2017 and provide the spatial-only baseline that was absent from the current evaluation.

5.4.2 Long-Term Research Avenues (3–5 Years)

Extend universality to additional modalities. The backbone-agnostic design of AnyUp3D is not limited to RGB video, and the zero-shot Swin-T result in Chapter 4 provides early empirical support for this generality. Extending the framework to handle features from depth sensors, LiDAR (Light Detection And Ranging) point clouds, or thermal cameras would broaden its applicability to autonomous driving and robotics perception pipelines, where multi-modal temporal consistency is a persistent open challenge.

Efficient temporal attention for real-time deployment. The current 3D cross-attention mechanism has quadratic complexity in the temporal dimension. Replacing it with linear attention or sparse windowed attention, analogous to the approach taken in Video Swin Transformer, would reduce inference time sufficiently to enable real-time processing at 30 frames per second on consumer-grade GPUs. This is a prerequisite for deployment in video editing tools and live streaming applications.

5.5 Final Concluding Remarks

This thesis demonstrated that the resolution gap inherent to Vision Foundation Models can be addressed in the temporal domain as well as the spatial domain. By introducing 3D convolutional encoding and 3D cross-attention with temporal masking into the AnyUp framework, AnyUp3D produces high-resolution feature maps that are stable across time, achieving a mean temporal cosine similarity improvement of +0.081 over frame-independent upsampling on every clip in the DAVIS-2017 benchmark, and translating this into a +0.074 improvement in downstream video object segmentation quality, evaluated zero-shot on Swin-T, a backbone the model never saw during training. Beyond the quantitative gains, a meaningful practical contribution is that AnyUp3D treats video as a first-class input. The original AnyUp must be applied frame-by-frame when used in a video pipeline, producing flickering artifacts that require external post-processing to suppress. AnyUp3D accepts a video tensor directly and handles temporal coherence internally, making it considerably easier to integrate into downstream video understanding systems without any additional stabilisation stages. This video-native design is itself a contribution: the model is strictly more capable than its predecessor in the video domain, independent of the specific numerical improvements reported here. The work establishes a principled, backbone-agnostic foundation for temporally consistent universal upsampling, a claim supported not only by design but by direct zero-shot evidence, confirming that explicitly modelling inter-frame dependencies is both necessary and sufficient to overcome the flickering artifacts that limit the practical utility of image-based upsampling methods in video applications.

REFERENCES

[1] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jégou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "Dinov2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.

[3] M. Hamilton, Z. Zhang, B. Hariharan, N. Snavely, and W. T. Freeman, "Featup: A model-agnostic framework for features at any resolution," in *International Conference on Learning Representations*, 2024.

[4] H. Huang, A. Chen *et al.*, "Loftup: Learning a coordinate-based feature upsampler for vision foundation models," *arXiv preprint arXiv:2504.14032*, 2025.

[5] T. Wimmer, P. Truong *et al.*, "Anyup: A universal feature upsampling model," *arXiv preprint arXiv:2510.12764*, 2025.

[6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, 2021.

[7] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[8] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[9] Y. Chen, S. Liu, and X. Wang, "Learning continuous image representation with local implicit image function," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.

[10] J. Wang, K. Chen *et al.*, "Carafe: Content-aware reassembly of features," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019.

[11] J. Yang *et al.*, "Denoising vision transformers," *arXiv preprint arXiv:2401.02957*, 2024.

[12] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. Schwing, "Cutie: A unified video object segmentation model with object memory," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[13] Z. Liu, J. Ning, Y. Cao, Y. Wei, Z. Zhang, S. Lin, and H. Hu, "Video swin transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[14] Z. Chen, Y. Zhang, X. Sun *et al.*, "Upscale-a-video: Temporal-consistent diffusion model for real-world video super-resolution," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[15] N. Ravi, A. Gabas, C. Rolland, C. Yu, C. Hu, J. Chu, S. Starke *et al.*, "Sam 2: Segment anything in images and videos," *arXiv preprint arXiv:2408.00714*, 2024.

[16] P. Couairon, L. Chambon, L. Serrano, J.-E. Haugeard, M. Cord, and N. Thome, "JAFAR: Jack up any feature at any resolution," in *Advances in Neural Information Processing Systems*, 2025.

[17] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A dataset of 101 human actions classes from videos in the wild," *arXiv preprint arXiv:1212.0402*, 2012.

[18] J. Pont-Tuset, F. Perazzi *et al.*, "The 2017 davis challenge on video object segmentation," *arXiv preprint arXiv:1704.00675*, 2017.

[19] Z. Tong, Y. Song, J. Wang, and L. Wang, "VideoMAE: Masked autoencoders are data-efficient learners for self-supervised video pre-training," in *NeurIPS*, 2022.

[20] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.

[21] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev *et al.*, "The kinetics human action video dataset," *arXiv preprint arXiv:1705.06950*, 2017.

[22] A. Jabri, A. Owens, and A. A. Efros, "Space-time correspondence as a contrastive random walk," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 19 545–19 560.

[23] S. W. Oh, J.-Y. Lee, N. Xu, and S. J. Kim, "Video object segmentation using space-time memory networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[24] H. K. Cheng, Y.-W. Tai, and C.-K. Tang, "Xmem: Long-term video object segmentation with an atkinson-shiffrin memory model," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.

[25] N. Xu, L. Yang, Y. Fan, D. Yang, Y. Yue, J. Liang, J. Yang, T. Huang, and T. S. Huang, "YouTube-VOS: A large-scale video object segmentation benchmark," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018.